FEM - Solution to Mechanical Equilibrium and Special Elements Sections

Vítor Azevedo

Contents

Introduction

This document seeks to organize the solutions to the chapters 4 and 5 of Finite Elements solutions course at *Universidade de Brasília*.

1. Theorical Solutions

1.1 Exercise 4.1.1

In which cases the strain tensor is symmetric?

The strain tensor is symmetric in all cases where the small displacement assumption holds, as it is defined as the symmetric part of the displacement gradient.

1.2 Exercise 4.1.2

Why is the stress tensor always symmetric?

It is a consequence of **conservation of angular momentum**. For example, σ_{xy} represents the stress acting on the x-axis, pointing to the y direction (and vice-versa). If $\sigma_{xy} \neq \sigma_{yx}$, this would imply the element has no **rotational equilibrium**, which, for the purposes of Linear Elastic analysis for small displacements, is false.

1.3 Exercise 4.1.3

Why does the classical finite element formulation assume small deformations?

The Green-Lagrange strain tensor can be defined as

$$\boldsymbol{E} = \frac{1}{2}(\boldsymbol{\nabla_X u} + \boldsymbol{\nabla_X u^T} + \boldsymbol{\nabla_X u^T \nabla_X u}) \tag{1.1}$$

Which, for small deformations, can be rounded to

$$\boldsymbol{E} \approx \frac{1}{2}(\boldsymbol{\nabla_X u} + \boldsymbol{\nabla_X u^T}) \tag{1.2}$$

Because the quadratic term can be neglected. The approximation above is the basis for linear elastic FEM analysis, and becomes false for fair deformations.

1.4 Exercise 4.1.4

Give examples of typical structures corresponding to each stress state: plane stress, plane strain, and axisymmetric conditions. Provide at least three examples for each case.

Plane Stress Conditions

- Thin plates with holes;
- Concrete walls, where its thickness is small compared to width and height.

Plane Strain Conditions

- Gravity dams;
- Retaining walls;

Axisymmetric Conditions

- Pressure vessels;
- Shafts and pipes.

1.5 Exercise 4.2.1

Explain the role of the strain-displacement matrix $\boldsymbol{B}$ in a finite element formulation.

The strain-displacement matrix $\boldsymbol{B}$ mathematically maps the discrete nodal displacements of an element to the continuous strain field within its domain.

That is, it relates the **global strains** to the **element's displacements**.

$$\boldsymbol{\varepsilon} = \boldsymbol{\partial u} = \boldsymbol{\partial}\left(\boldsymbol{N U}^{(e)}\right) = (\boldsymbol{\partial N})\boldsymbol{U}^{(e)} = \boldsymbol{B U}^{(e)} \tag{1.3}$$

Thus, $\boldsymbol{B} = \boldsymbol{\partial N}$ contains the spatial derivatives of the element shape functions.

It also plays extremely important roles in the stiffness matrix definition and calculation, and represents a way to obtain strains and stresses when the displacements are known.

1.6 Exercise 4.3.1

What is an equivalent nodal-load vector, and why is it needed in the finite element method?

An **equivalent nodal-load vector** $\boldsymbol{F}_{eq}^{(e)}$ is a discrete vector that represents non-nodal external actions —such as distributed body forces (gravity, self-weight, eletromagnetic forces, that act on the **Volume**), surface tractions (e.g., pressure, friction, that act on the **boundaries**), or thermal stresses—acting on an element as statically equivalent forces acting exclusively at its nodes, calculated via the **Principle of Virtual Work**.

It is absolutely essential in the Finite Element Method for composing the vector $\boldsymbol{F}$ at the main FEM linear system ($\boldsymbol{KU} = \boldsymbol{F}$).

$$\boldsymbol{F}_b^{(e)} = \int_{\Omega^{(e)}} \boldsymbol{N}^T \boldsymbol{b} dV, \qquad \boldsymbol{F}_t^{(e)} = \int_{\Gamma_t^{(e)}} \boldsymbol{N}^T \boldsymbol{t} dS \tag{1.4}$$

This consistency guarantees that the finite element solution minimizes the potential energy error of the structural system.

1.7 Exercise 4.3.2

Explain the difference between an externally applied nodal force and a consistent equivalent nodal load obtained from a distributed action.

Externally Applied Nodal Force: This represents a true concentrated action (say a cord that exerts traction in a sufficiently small area) that is applied directly to a specific, existing node in the finite element mesh. (note that it then requires a node at that specific location, during pre-processing). They are directly inserted into the $\boldsymbol{F}$ vector

Consistent Equivalent Nodal Load: This is a translation of a distributed action that acts over an entire element domain, rather than at a single point. Examples include body forces (like self-weight/ gravity acting on volume) or surface tractions (like wind or hydrostatic pressure acting on an element's face). They require the use of integrals to properly "wage" the impacts of the load into each node of the element, which can bring erros if the mesh is not adequate.

1.8 Exercise 4.3.3

How do body forces and surface tractions enter the weak form and the element load vector?

- **body forces**
  - Body forces are applied to the element's volume (via the material's γ property, that relates force per volume element). Thus, they are calculated via the following integral:

$$F_{bf} = \int_{\boldsymbol{\Omega}^{(e)}} \boldsymbol{N}^T \boldsymbol{b} dV \tag{1.5}$$

    Where $\boldsymbol{N}$ is the shape functions matrix, and $\boldsymbol{b}$ the force factor each element of volume applies to the element.

- **surface tractions**
  - Surface tractions, on the other hand, are applied to the element's surfaces, that is, they are external to the element, and represent actions done by other structures onto the Dominium of analysis $\boldsymbol{\Omega}^{(e)}$. These applied forces are defined by the traction vector $\boldsymbol{t}$ They can be calculated by

$$F_t = \int_{\boldsymbol{\Gamma}^{(e)}} \boldsymbol{N}^T \boldsymbol{t} dS \tag{1.6}$$

As we can see, the shape functions matrix, defined as

$$\boldsymbol{N} = [N_1\boldsymbol{I}, N_2\boldsymbol{I}, N_3\boldsymbol{I}, ..., N_i\boldsymbol{I}] \tag{1.7}$$

Is the key to make the distribution of otherwise unknown forces into nodal forces

1.9 Exercise 4.4.1

What is full and reduced integration in the computation of the stiffness matrix?

- **Full Integration**: This refers to using the standard or mathematically required number of integration points (Gauss points) to integrate the terms in the stiffness matrix exactly (or up to the degree of exactness required by the order of the interpolating polynomial). As seen, a n number gaussian quadrature will perfectly solve polynomials of up to $2n-1$ degree
- **Reduced Integration**: This technique uses a lower number of integration points than what is required for full integration, which reduces computacional time at the cost of accuracy. Note that as shown in the Numerical issues section, because of shear locking, reduced integrations can approximate to the real solution better.

1.10 Exercise 4.4.2

What are the connectivity and location vectors of an element, and how are they used in the assembly process?

The connectivity vector maps an element's local nodes to their global node numbers, defining structural topology. The location vector (LM) maps the element's local degrees of freedom (DOFs) directly to their corresponding global DOF indices. In the assembly process, the location vector acts as an indexing guide to distribute and superimpose the local element stiffness terms ($\boldsymbol{K}^{(e)}$) into the correct rows and columns of the global stiffness matrix ($\boldsymbol{K}^{(g)}$) using the principle of superposition.

1.11 Exercise 4.4.3

What is the equilibrium residual, and why must it vanish only at free degrees of freedom?

The equilibrium residual is the difference between external and internal forces, which, for a converged static solution, should be null at free degrees of freedom. Such As

$$R = F_{\text{ext}} - F_{\text{int}} \qquad R_{\text{fdof}} \approx 0 \tag{1.8}$$

If the residual at a free degree of freedom is not small, the finite-element equations have not been satisfied there. For a linear elastic analysis, this indicates an inconsistent load vector, or incorrect assembly of stiffness matrix.

1.12 Exercise 4.4.4

How are support reactions recovered after the displacement field has been computed?

Support reactions are obtained with the residual, as seen in the last problem. Supports are restricted degrees of freedom, and their reactions represent the necessary conditions to maintain mechanical equilibrium through the element. Thus, the reaction vector can be defined as

$$\boldsymbol{r}_c = \boldsymbol{F}_{\text{int}_c} - \boldsymbol{F}_{\text{ext}_c} \tag{1.9}$$

1.13 Exercise 4.5.1

Explain the origin of shear locking and volumetric locking, and indicate in which types of problems each one is most likely to appear.

- **Shear locking** arises when structural elements (like Timoshenko beams or Mindlin plates) cannot undergo pure bending without generating parasitic shear strains, causing the element to behave with artificial stiffness. It is most common in very thin or slender elements.
- **Volumetric locking** occurs when elements cannot properly represent isochoric (volume-preserving) deformations without generating spurious volumetric strains. It typically appears in problems involving

nearly incompressible materials (e.g., rubber, where Poisson's ratio approaches 0.5) or in fully plastic deformation scenarios.

1.14 Exercise 4.5.2

Why can reduced integration alleviate locking, and what numerical risk does it introduce?

Reduced integration evaluates the specific terms causing artificial stiffness (such as transverse shear or volumetric strain energy) at fewer integration points, effectively relaxing the overly strict kinematic constraints. However, the numerical risk it introduces is rank deficiency in the stiffness matrix. This can allow non-physical, zero-energy deformation modes to develop and entirely corrupt the numerical solution.

1.15 Exercise 4.5.3

What are hourglass modes, and why are they associated with underintegrated elements?

Hourglass modes are spurious, non-physical deformation patterns that produce zero strain at the integration points, meaning the element deforms without storing any strain energy. They are uniquely associated with underintegrated elements because the reduced number of Gauss points fails to detect the strains generated by these specific zig-zag or warping shapes, resulting in zero stiffness being assigned to those modes.

1.16 Exercise 4.5.4

Why can element distortion degrade accuracy even when the interpolation order is unchanged?

Severe element distortion alters the mapping between the natural (parametric) and physical coordinate systems, causing the Jacobian determinant to vary drastically across the element or approach zero. This poor mapping skews the shape function derivatives and drastically degrades the accuracy of the numerical integration, leading to significant discretization errors regardless of the element's theoretical polynomial order.

1.17 Exercise 4.5.5

What is meant by ill-conditioning of the stiffness matrix, and how can it affect the numerical solution?

Ill-conditioning means the stiffness matrix has a very large condition number, reflecting an extreme disparity between its largest and smallest eigenvalues (often due to huge differences in element stiffnesses). This affects the numerical solution by making the system highly sensitive to floating-point round-off errors, which can result in severely inaccurate displacement values or cause iterative solvers to fail to converge.

1.18 Exercise 4.5.6

Distinguish between an ill-conditioned stiffness matrix and a singular stiffness matrix.

A singular stiffness matrix has a determinant of exactly zero, meaning it cannot be inverted at all and the structural system has no unique solution (typically indicating unrestrained rigid body motion). An ill-conditioned stiffness matrix is mathematically invertible, but the extreme variation in its numerical values makes the inversion process highly unstable and prone to massive round-off errors.

1.19 Exercise 4.5.7

Give four common modeling or formulation errors that can lead to a singular stiffness matrix.

Four common errors include:

- (1) insufficient boundary conditions that leave the structure with unrestrained rigid body modes;
- (2) internal mechanisms, such as collinear truss elements without transverse support or unresisted hinges;
- (3) disconnected elements or unmerged nodes that create floating, independent parts;
- (4) missing material or geometric properties (e.g., zero Young's modulus or zero area) yielding zero-stiffness entries.

1.20 Exercise 4.5.8

Why may excessively large penalty parameters create numerical difficulties even though they enforce constraints more strongly?

Excessively large penalty parameters introduce values into the global stiffness matrix that are many orders of magnitude larger than the actual physical stiffness terms of the elements. This massive disparity causes severe ill-conditioning; during the matrix solution process, floating-point arithmetic limits cause the smaller, genuine physical stiffness data to be truncated or lost entirely, leading to numerical instability and highly inaccurate results.

1.21 Exercise 4.6.1

List five use-cases of multifreedom constraints in finite-element modeling.

Multifreedom constraints (MFCs) are typically used to model situations where a displacement component cannot be prescribed independently. Typical use-cases include:

1. **Inclined supports**.
2. **Rigid links or rigid inclusions**.
3. **Symmetry relations**.
4. **Tied interfaces**.
5. **Contact ties**.

1.22 Exercise 4.6.2

Compare master-slave elimination, the penalty method, and the Lagrange multiplier method for enforcing constraints. State one advantage and one drawback of each.

Master-slave elimination:

- **Advantage: Enforces constraints exactly** and does not add new unknowns to the system.
- **Drawback: Implementation becomes complex** for numerous constraints because the global numbering and transformation matrices must be handled consistently.

Penalty method:

- **Advantage: Simple to implement** and maintains the positive definiteness of the stiffness matrix without expanding its size.
- **Drawback: Requires an artificial penalty parameter**; excessively large values lead to severe ill-conditioning, while small values fail to strictly enforce the constraint.

Lagrange multiplier method:

- **Advantage: Enforces constraints exactly** at the discrete level and computes the exact constraint (reaction) forces automatically as a byproduct.
- **Drawback: Increases the overall size of the global system** by adding new unknowns and makes the matrix indefinite by introducing zero-diagonal blocks, which requires more complex solvers.

1.23 Exercise 5.1.1

List four applications for each of the following element types: plate, membrane, shell, and interface.

- **Plate elements:** Slabs, bridge decks, floor panels, and structures where one dimension is much smaller than the other two.
- **Membrane elements:** Reinforcement sheets, geotextiles, tensile roofs, and embedded reinforcement.
- **Shell elements:** Pressure vessels, silos, aircraft fuselages, and thin-walled pipes.
- **Interface elements:** Joints, bonded interfaces, contact zones, and potential crack paths.

2. Exercise 4.1.5

A flat metallic plate with length 100 mm, width 40 mm, and thickness 5 mm is subjected to a uniform tensile load. When the axial force was measured as 20 kN the axial elongation was 0.1 mm and in-plane lateral contraction 0.008 mm. Determine the corresponding constitutive matrix **D** for a plane stress 2D linear elastic model.

2.1 Solution

Firstly, we need to calculate the tensions

$$\sigma_x = \frac{F}{A} = \frac{20000 \text{ N}}{0.0002\text{m}^2} = 100 \text{ MPa} \tag{4.1.5.1}$$

$$\varepsilon_x = \frac{\Delta L}{L} = \frac{0.0001 \text{ m}}{0.1 \text{ m}} = 0.001 \text{ m/m} \tag{4.1.5.2}$$

It is known that $E = \frac{\sigma}{\varepsilon}$, so:

$$E = \frac{100000000}{0.001} = 100000000000 \text{ Pa} = 100 \text{ GPa} \tag{4.1.5.3}$$

Similarly

$$\varepsilon_y = \frac{\Delta W}{W} = \frac{-0.000008}{0.04} = -0.0002 \tag{4.1.5.4}$$

$$\nu = -\frac{\varepsilon_y}{\varepsilon_x} = \frac{-0.0002}{0.001} = 0.2 \tag{4.1.5.5}$$

The constitutive Matrix is then defined by

$$\boldsymbol{D} = \frac{E}{1-\nu^2} \cdot \begin{pmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & \frac{1-\nu}{2} \end{pmatrix} \tag{4.1.5.6}$$

so

$$\boldsymbol{D} = \frac{100000000000}{1-(0.2)^2} \cdot \begin{bmatrix} 1 & 0.2 & 0 \\ 0.2 & 1 & 0 \\ 0 & 0 & \frac{1-(0.2)}{2} \end{bmatrix} \tag{4.1.5.7}$$

$$\boldsymbol{D} = 104.1667\text{GPa} \cdot \begin{bmatrix} 1 & 0.2 & 0 \\ 0.2 & 1 & 0 \\ 0 & 0 & 0.4 \end{bmatrix} \tag{4.1.5.8}$$

$$\boldsymbol{D} = \begin{bmatrix} 104.1667 & 20.8333 & 0 \\ 20.8333 & 104.1667 & 0 \\ 0 & 0 & 41.6667 \end{bmatrix} \text{ GPa} \tag{4.1.5.9}$$

---

3. Exercise 4.1.6

A cylindrical concrete sample with height 20 cm and diameter 10 cm is tested in uniaxial compression. At a point within the elastic range, the measured values are: axial stress 20 MPa, axial shortening 0.2 mm, and radial expansion 0.015 mm. Using these measurements, determine the elastic parameters E and ν, and construct the corresponding constitutive matrix $\boldsymbol{D}$ for a plane stress linear[1] elastic model.

3.1 Solution

Firstly, we need to define Young's module:

$$E = \frac{\sigma}{\varepsilon} \tag{4.1.6.1}$$

$$\varepsilon = \frac{\Delta L}{L} \tag{4.1.6.2}$$

Substituting (4.1.6.2) in (4.1.6.1), we get

$$E = \frac{\sigma \cdot L}{\Delta L} = \frac{20\ \text{MPa} \cdot 20\ \text{cm}}{0.02\ \text{cm}} = 20\ \text{GPa} \tag{4.1.6.3}$$

Now, we may obtain epsilon in relation to the radius:

$$\varepsilon_z = \frac{\Delta L}{L} = -0.001\ \text{m/m} \tag{4.1.6.4}$$

$$\varepsilon_r = \frac{\Delta r}{r} = 0.0003\ \text{m/m} \tag{4.1.6.5}$$

Defining Poisson:

$$\nu = -\frac{\varepsilon_r}{\varepsilon_z} = 0.3 \tag{4.1.6.6}$$

Firstly, let's analyse the constitutive matrix for an axissimetrical element, defined as

$$\begin{bmatrix} \sigma_{\text{rr}} \\ \sigma_{\text{zz}} \\ \sigma_{\theta\theta} \\ \tau_{rz} \end{bmatrix} = \boldsymbol{D} \cdot \begin{bmatrix} \varepsilon_{\text{rr}} \\ \varepsilon_{\text{zz}} \\ \varepsilon_{\theta\theta} \\ \gamma_{rz} \end{bmatrix} \tag{4.1.6.7}$$

Where **D** is defined by

$$\boldsymbol{D} = \frac{E}{(1+\nu)(1-2\nu)} \cdot \begin{bmatrix} 1-\nu & \nu & \nu & 0 \\ \nu & 1-\nu & \nu & 0 \\ \nu & \nu & 1-\nu & 0 \\ 0 & 0 & 0 & \frac{1-2\nu}{2} \end{bmatrix} \tag{4.1.6.8}$$

$$\boldsymbol{D} = 38461.5385\ \text{MPa} \cdot \begin{bmatrix} 0.7 & 0.3 & 0.3 & 0 \\ 0.3 & 0.7 & 0.3 & 0 \\ 0.3 & 0.3 & 0.7 & 0 \\ 0 & 0 & 0 & 0.2 \end{bmatrix} \tag{4.1.6.9}$$

[1] I've read the problem incorrectly and done also the axissimetrical development, oops!

$$= \begin{bmatrix} 26923.1 & 11538.5 & 11538.5 & 0 \\ 11538.5 & 26923.1 & 11538.5 & 0 \\ 11538.5 & 11538.5 & 26923.1 & 0 \\ 0 & 0 & 0 & 7692.3 \end{bmatrix} \text{ MPa} \tag{4.1.6.10}$$

Finally, to define the same element in a plane stress linear elastic model:

$$\boldsymbol{D} = \frac{E}{1-\nu^2} \begin{pmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & \frac{1-\nu}{2} \end{pmatrix} \tag{4.1.6.11}$$

$$\boldsymbol{D} = \frac{20000 \text{ MPa}}{1-0.3^2} \cdot \begin{bmatrix} 1 & 0.3 & 0 \\ 0.3 & 1 & 0 \\ 0 & 0 & 0.35 \end{bmatrix} \tag{4.1.6.12}$$

$$\boldsymbol{D} = \begin{bmatrix} 2.1978 & 0.6593 & 0 \\ 0.6593 & 2.1978 & 0 \\ 0 & 0 & 0.7692 \end{bmatrix} \cdot 10^4 \text{ MPa} \tag{4.1.6.13}$$

---

4. Exercise 4.1.7

A long thick-walled cylindrical pipe with inner radius a = 50 mm and outer radius b = 100 mm is subjected to a uniform internal pressure p = 10 MPa. The material is linear elastic and isotropic, with Young's modulus E = 200 GPa and Poisson's ratio v = 0.2. Assume that the pipe is sufficiently long so that the axial strain can be neglected, i.e. εz = 0. At a point located at r = 75 mm within the wall, the stress state is characterized by normal stresses only, with no shear components. Determine the stress components at this point and the corresponding strain components.

4.1 Solution

Since the internal pressure acts in the radial direction, and assuming axial strength can be neglected, it is valid to assume that the problem constitutes an Axisymmetric Stress, defined by

$$\boldsymbol{\sigma} = \begin{bmatrix} \sigma_{rr} & 0 & \sigma_{rz} \\ 0 & \sigma_{\theta\theta} & 0 \\ \sigma_{zr} & 0 & \sigma_{zz} \end{bmatrix} \tag{4.1.7.1}$$

It is known, for the current material, that

$$\nu = 0.2 \tag{4.1.7.2}$$

$$E = 200 \text{ MPa} \tag{4.1.7.3}$$

Thus, for the Axisymmetric condition:

$$\begin{bmatrix} \sigma_{\text{rr}} \\ \sigma_{\text{zz}} \\ \sigma_{\theta\theta} \\ \tau_{rz} \end{bmatrix} = \boldsymbol{D} \cdot \begin{bmatrix} \varepsilon_{\text{rr}} \\ \varepsilon_{\text{zz}} \\ \varepsilon_{\theta\theta} \\ \gamma_{rz} \end{bmatrix} \tag{4.1.7.4}$$

But, since axial strain can be neglected, that is, $\varepsilon_{zz} = 0$, we can define it as

$$\begin{bmatrix} \sigma_{\text{rr}} \\ \sigma_{\theta\theta} \\ \tau_{rz} \end{bmatrix} = \boldsymbol{D} \cdot \begin{bmatrix} \varepsilon_{\text{rr}} \\ \varepsilon_{\theta\theta} \\ \gamma_{rz} \end{bmatrix} \tag{4.1.7.5}$$

Where $\boldsymbol{D}$, in this specific case, will be

$$\boldsymbol{D} = \frac{E}{(1+\nu)(1-2\nu)} \cdot \begin{bmatrix} 1-\nu & \nu & 0 \\ \nu & 1-\nu & 0 \\ 0 & 0 & \frac{1-2\nu}{2} \end{bmatrix} \tag{4.1.7.6}$$

$$\boldsymbol{D} = \frac{200}{(1+0.2)(1-0.4)} \cdot \begin{bmatrix} 0.8 & 0.2 & 0 \\ 0.2 & 0.8 & 0 \\ 0 & 0 & 0.3 \end{bmatrix} \tag{4.1.7.7}$$

$$\boldsymbol{D} = \begin{bmatrix} 222.2222 & 55.5556 & 0 \\ 55.5556 & 222.2222 & 0 \\ 0 & 0 & 83.3333 \end{bmatrix} \text{ MPa} \tag{4.1.7.8}$$

With the defined matrix:[2]

[2]This problem is incomplete

5. Exercise 4.2.2

Build the matrix $\boldsymbol{B}$ for a three-node bar element with nodal coordinates $x1$, $x2$, and $x3$.

5.1 Solution

It is known that

$$u = \sum_{i=1}^{3} N_i(\xi) u_i = \boldsymbol{N}\boldsymbol{U}^e \tag{4.1.7.1}$$

$$\varepsilon = \frac{du}{dx} = \frac{d\boldsymbol{N}^T}{dx}\boldsymbol{U}^e \tag{4.1.7.2}$$

$$\frac{d\boldsymbol{N}^T}{dx} = \frac{d\boldsymbol{N}^T}{d\xi} \cdot J^{-1} \tag{4.1.7.3}$$

$$\varepsilon = \frac{du}{dx} = \frac{1}{J}\frac{d\boldsymbol{N}^T}{d\xi}\boldsymbol{U}^e \tag{4.1.7.4}$$

Thus

$$\begin{aligned} \boldsymbol{B} &= \frac{1}{J}\frac{d\boldsymbol{N}^T}{d\xi} \\ &= \frac{1}{J}\begin{pmatrix} \frac{dN_1}{d\xi} & \frac{dN_2}{d\xi} & \frac{dN_3}{d\xi} \end{pmatrix} \end{aligned} \tag{4.1.7.5}$$

For a 3-node bar element

$$\boldsymbol{N}^T = \begin{pmatrix} \frac{\xi(\xi-1)}{2} & \frac{\xi(\xi+1)}{2} & 1-\xi^2 \end{pmatrix} \tag{4.1.7.6}$$

$$\frac{d\boldsymbol{N}^T}{d\xi} = \begin{pmatrix} \frac{2\xi-1}{2} & \frac{2\xi+1}{2} & -2\xi \end{pmatrix} \tag{4.1.7.7}$$

$$J = \boldsymbol{X}^T\frac{d\boldsymbol{N}}{d\xi} \tag{4.1.7.8}$$

Thus

$$\boldsymbol{B} = \left(\boldsymbol{X}^T\frac{d\boldsymbol{N}}{d\xi}\right)^{-1}\frac{d\boldsymbol{N}^T}{d\xi} \tag{4.1.7.9}$$

For

$$\boldsymbol{X} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} \tag{4.1.7.10}$$

6. Exercise 4.2.3

Find the matrix $\boldsymbol{B}$ for a three-node triangular element with nodal coordinates (x_1, y_1), (x_2, y_2), and (x_3, y_3). Why is the matrix $\boldsymbol{B}$ constant along the element, and what does that imply?

6.1 Solution

First of all, a 3 node triangular element is defined by the following shape functions

$$\boldsymbol{N} = \begin{pmatrix} 1-\xi-\eta \\ \xi \\ \eta \end{pmatrix} \tag{4.2.3.1}$$

For

$$\boldsymbol{x} = \sum_{i=1}^{3} N_i(\xi,\eta)\cdot \boldsymbol{x}_i \tag{4.2.3.2}$$

$$\boldsymbol{u} = \sum_{i=1}^{3} N_i(\xi,\eta)\cdot \boldsymbol{u}_i \tag{4.2.3.3}$$

$$\begin{bmatrix} \varepsilon_{xx} \\ \varepsilon_{yy} \\ \gamma_{xy} \end{bmatrix} = \boldsymbol{B}\cdot \begin{bmatrix} u_{1x} \\ u_{1y} \\ u_{2x} \\ u_{2y} \\ u_{3x} \\ u_{3y} \end{bmatrix} \tag{4.2.3.4}$$

$$\boldsymbol{B} = (\boldsymbol{B}_1 \;\; \boldsymbol{B}_2 \;\; \boldsymbol{B}_3) \tag{4.2.3.5}$$

$$\boldsymbol{B}_i = \begin{pmatrix} \frac{\partial N_i}{\partial x} & 0 \\ 0 & \frac{\partial N_i}{\partial y} \\ \frac{\partial N_i}{\partial y} & \frac{\partial N_i}{\partial x} \end{pmatrix} \tag{4.2.3.6}$$

It is known that

$$\frac{\partial \boldsymbol{N}_i}{\partial \boldsymbol{x}} = \frac{\partial \boldsymbol{N}_i}{\partial \boldsymbol{\xi}} \boldsymbol{J}^{-1} \tag{4.2.3.7}$$

$$x(\xi,\eta) = (1-\xi-\eta)x_1 + \xi x_2 + \eta x_3 = x_1 + \xi(x_2-x_1) + \eta(x_3-x_1) \tag{4.2.3.8}$$

$$y(\xi,\eta) = (1-\xi-\eta)y_1 + \xi y_2 + \eta y_3 = y_1 + \xi(y_2-y_1) + \eta(y_3-y_1) \tag{4.2.3.9}$$

Thus the Jacobian is

$$\boldsymbol{J} = \begin{pmatrix} x_2-x_1 & y_2-y_1 \\ x_3-x_1 & y_3-y_1 \end{pmatrix} = \begin{pmatrix} x_{21} & y_{21} \\ x_{31} & y_{31} \end{pmatrix} \tag{4.2.3.10}$$

$$J = \det(\boldsymbol{J}) = x_{21}y_{31} - x_{31}y_{21} = (x_2-x_1)(y_3-y_1) - (x_3-x_1)(y_3-y_1) \tag{4.2.3.11}$$

Which, as seen by the shoelace formula, $J = 2A$.

It is known that

$$\boldsymbol{J}^{-1} = \frac{\operatorname{adj}(\boldsymbol{J})}{\det(\boldsymbol{J})} = \frac{1}{2A}\begin{pmatrix} y_{31} & -y_{21} \\ -x_{31} & x_{21} \end{pmatrix} \tag{4.2.3.12}$$

For the first node

$$\begin{pmatrix} \frac{\partial N_1}{\partial x} \\ \frac{\partial N_1}{\partial y} \end{pmatrix} = \boldsymbol{J}^{-1}\begin{pmatrix} \frac{\partial N_1}{\partial \xi} \\ \frac{\partial N_1}{\partial \eta} \end{pmatrix} = \frac{1}{2A}\begin{pmatrix} y_{31} & -y_{21} \\ -x_{31} & x_{21} \end{pmatrix}\begin{pmatrix} -1 \\ -1 \end{pmatrix} \tag{4.2.3.13}$$

$$\begin{pmatrix} \frac{\partial N_1}{\partial x} \\ \frac{\partial N_1}{\partial y} \end{pmatrix} = \frac{1}{2A}\begin{pmatrix} y_{21} - y_{31} \\ x_{31} - x_{21} \end{pmatrix} = \frac{1}{2A}\begin{pmatrix} y_{23} \\ x_{32} \end{pmatrix} \tag{4.2.3.14}$$

For the second node

$$\begin{pmatrix} \frac{\partial N_2}{\partial x} \\ \frac{\partial N_2}{\partial y} \end{pmatrix} = \boldsymbol{J}^{-1}\begin{pmatrix} \frac{\partial N_2}{\partial \xi} \\ \frac{\partial N_2}{\partial \eta} \end{pmatrix} = \frac{1}{2A}\begin{pmatrix} y_{31} & -y_{21} \\ -x_{31} & x_{21} \end{pmatrix}\begin{pmatrix} 1 \\ 0 \end{pmatrix} \tag{4.2.3.15}$$

$$\begin{pmatrix} \frac{\partial N_2}{\partial x} \\ \frac{\partial N_2}{\partial y} \end{pmatrix} = \frac{1}{2A}\begin{pmatrix} y_{31} \\ x_{13} \end{pmatrix} \tag{4.2.3.16}$$

For the third node

$$\begin{pmatrix} \frac{\partial N_3}{\partial x} \\ \frac{\partial N_3}{\partial y} \end{pmatrix} = \frac{1}{2A}\begin{pmatrix} y_{12} \\ x_{21} \end{pmatrix} \tag{4.2.3.17}$$

Thus, finally:

$$\boldsymbol{B} = \frac{1}{2A} \cdot \begin{pmatrix} y_{23} & 0 & y_{31} & 0 & y_{12} & 0 \\ 0 & x_{32} & 0 & x_{13} & 0 & x_{21} \\ x_{32} & y_{23} & x_{13} & y_{31} & x_{21} & y_{12} \end{pmatrix} \tag{4.2.3.18}$$

Note the notation $x_{ij} = x_i - x_j$.

The strain-displacement matrix $\boldsymbol{B}$ is uniform across the entire domain of the element Ω_e:

$$\boldsymbol{B}(x, y) = \text{const} \quad \forall (x, y) \in \Omega_e \tag{4.2.3.19}$$

This characteristic is true because the Jacobian J is constant, because the element is linear.

7. Exercise 4.2.4

For the quadrilateral element shown below, and given its nodal coordinates and nodal-displacement vector

$$C = \begin{pmatrix} 0 & 0 \\ 4 & 2 \\ 4 & 4 \\ 0 & 2 \end{pmatrix} \text{ m} \quad \text{and} \quad U^{(e)} = \begin{pmatrix} 0 \\ 0 \\ 0.01 \\ 0.01 \\ 0.015 \\ 0.015 \\ 0 \\ 0.015 \end{pmatrix} \text{ m} \tag{4.2.4.1}$$

as shown in the figure. Approximate the strain vector at the natural coordinates $(\xi, \eta) = \left(-\sqrt{\frac{1}{3}}, \sqrt{\frac{1}{3}}\right)$.

7.1 Solution

First step, is to define the shape functions and its Jacobian

$$\boldsymbol{N}(\xi, \eta) = \begin{pmatrix} \frac{(1-\xi)(1-\eta)}{4} \\ \frac{(1+\xi)(1-\eta)}{4} \\ \frac{(1+\xi)(1+\eta)}{4} \\ \frac{(1-\xi)(1+\eta)}{4} \end{pmatrix} \tag{4.2.4.2}$$

Or simply

$$N_i(\xi, \eta) = \frac{1}{4}(\xi_i - \xi)(\eta_i - \eta) \tag{4.2.4.3}$$

Taking its derivatives

$$\begin{pmatrix} \frac{\partial N_i}{\partial \xi} \\ \frac{\partial N_i}{\partial \eta} \end{pmatrix} = \begin{pmatrix} \eta - \eta_i \\ \xi - \xi_i \end{pmatrix} \tag{4.2.4.4}$$

$$\begin{pmatrix} x(\xi, \eta) \\ y(\xi, \eta) \end{pmatrix} = \sum_{i=1}^{n} \begin{pmatrix} \frac{x_i}{4}(\xi_i - \xi)(\eta_i - \eta) \\ \frac{y_i}{4}(\xi_i - \xi)(\eta_i - \eta) \end{pmatrix} \tag{4.2.4.5}$$

The strain vector for a two dimensional element, which can suffer strains in xx and yy, and the shear strain xy, is defined by

$$\boldsymbol{\varepsilon} = (\boldsymbol{B}_1 \ \ \boldsymbol{B}_2 \ \ \boldsymbol{B}_3 \ \ \boldsymbol{B}_4) \cdot \boldsymbol{U}^e \tag{4.2.4.6}$$

Since we already know $\boldsymbol{U}^e$, we just have to use $\boldsymbol{C}$ to define $\boldsymbol{B}_i$:

$$\boldsymbol{B}_i = \begin{pmatrix} \frac{\partial N_i}{\partial x} & 0 \\ 0 & \frac{\partial N_i}{\partial y} \\ \frac{\partial N_i}{\partial y} & \frac{\partial N_i}{\partial x} \end{pmatrix} \tag{4.2.4.7}$$

$$\frac{\partial \boldsymbol{N}_i}{\partial \boldsymbol{x}} = \frac{\partial \boldsymbol{N}_i}{\partial \boldsymbol{\xi}} \boldsymbol{J}^{-1} \tag{4.2.4.8}$$

For $(\xi, \eta) = \left(-\frac{\sqrt{3}}{3}, \frac{\sqrt{3}}{3}\right)$, we have:

$$\frac{\partial \boldsymbol{N}}{\partial \boldsymbol{\xi}} = \begin{bmatrix} -0.1057 & -0.3943 \\ 0.1057 & -0.1057 \\ 0.3943 & 0.1057 \\ -0.3943 & 0.3943 \end{bmatrix} \tag{4.2.4.9}$$

And the Jacobian is:

$$\boldsymbol{J} = \begin{bmatrix} 2 & 0 \\ 1 & 1 \end{bmatrix} \tag{4.2.4.10}$$

Thus its inverse is:

$$\boldsymbol{J}^{-1} = \begin{bmatrix} 0.5 & 0 \\ -0.5 & 1 \end{bmatrix} \tag{4.2.4.11}$$

With (4.2.4.9) and (4.2.4.11) defined, it is simple to obtain the $\boldsymbol{B}$:

$$\boldsymbol{B} = \begin{bmatrix} 0.1443 & 0 & 0.1057 & 0 & 0.1443 & 0 & -0.3943 & 0 \\ 0 & -0.3943 & 0 & -0.1057 & 0 & 0.1057 & 0 & 0.3943 \\ -0.3943 & 0.1443 & -0.1057 & 0.1057 & 0.1057 & 0.1443 & 0.3943 & -0.3943 \end{bmatrix} \tag{4.2.4.12}$$

Substituting this into (4.2.4.6), with the use of Julia, we obtain the resultant ε matrix:

```
using LinearAlgebra

C = [
    0 0;
    4 2;
    4 4;
    0 2;
]

U = [
    0;
    0;
    0.01;
    0.01;
    0.015;
    0.015;
    0;
    0.015;
]


function N_derivs(xi, eta)
    return [
        0.25*(-1.0 + eta)  0.25*(-1.0 + xi)
        0.25*(+1.0 - eta)  0.25*(-1.0 - xi)
        0.25*(+1.0 + eta)  0.25*(+1.0 + xi)
        0.25*(-1.0 - eta)  0.25*(+1.0 - xi)
    ]
end

function Jacobian(C, xi, eta)
    # Get the 4x2 matrix of shape function derivatives
    dNdξ = N_derivs(xi, eta)
```

```
    return C' * dNdξ
end

function getB(C, xi, eta)
    dNdx = N_derivs(xi, eta) * inv(Jacobian(C, xi, eta))
    B = nothing
    for row in eachrow(dNdx)
        dx, dy = row
        Bi = [
            dx 0;
            0 dy;
            dy dx;
        ]
        if B === nothing
            B = Bi
        else
            B = [B Bi]
        end
    end

    return B
end

B = getB(C, -sqrt(1/3), sqrt(1/3))

ϵ = B * U

ϵ
```

```
3-element Vector{Float64}:
  0.003221687836487032
  0.006443375672974064
 -0.002165063509461096
```

Hence

$$\varepsilon = \begin{bmatrix} 0.003221688 \\ 0.006443376 \\ -0.002165064 \end{bmatrix} \tag{4.2.4.13}$$

8. Exercise 4.2.5

Find an expression for the matrix B of a three-node bar element in 2D as a function of the Jacobian and the local coordinate ξ. Consider $\varepsilon = \frac{\partial u}{\partial s}$, where s is the curvilinear coordinate along the bar path.

8.1 Solution

Strain is defined as the derivative of displacement. Because the 1D bar is embedded in 2D space, the structural axial strain ε is the projection of the global displacement derivatives onto the local unit tangent vector $\hat{\boldsymbol{r}}$:

$$\varepsilon = \hat{\boldsymbol{r}}^T \frac{\partial \boldsymbol{u}}{\partial s} \tag{4.2.5.1}$$

We know the scalar Jacobian relates the curvilinear coordinate s to the natural coordinate ξ:

$$J = \sqrt{\left(\frac{\partial x}{\partial \xi}\right)^2 + \left(\frac{\partial y}{\partial \xi}\right)^2} = \frac{\partial s}{\partial \xi} \tag{4.2.5.2}$$

Using the chain rule, the global displacement derivatives with respect to s are:

$$\frac{\partial \boldsymbol{u}}{\partial s} = \frac{\partial \boldsymbol{u}}{\partial \xi}\frac{\partial \xi}{\partial s} = \frac{1}{J}\frac{\partial \boldsymbol{u}}{\partial \xi} \tag{4.2.5.3}$$

The displacement derivatives with respect to ξ can be interpolated using the shape functions:

$$\frac{\partial \boldsymbol{u}}{\partial \xi} = \frac{\partial \boldsymbol{N}}{\partial \xi}\boldsymbol{U}^e \tag{4.2.5.4}$$

Where, for a 3-node bar in 2D, the shape function derivative matrix and the nodal displacement vector are:

$$\begin{gathered}\frac{\partial \boldsymbol{N}}{\partial \xi} = \begin{pmatrix} \frac{\partial N_1}{\partial \xi} & 0 & \frac{\partial N_2}{\partial \xi} & 0 & \frac{\partial N_3}{\partial \xi} & 0 \\ 0 & \frac{\partial N_1}{\partial \xi} & 0 & \frac{\partial N_2}{\partial \xi} & 0 & \frac{\partial N_3}{\partial \xi} \end{pmatrix} \\ \boldsymbol{U}^e = \begin{pmatrix} u_1 \\ v_1 \\ u_2 \\ v_2 \\ u_3 \\ v_3 \end{pmatrix}\end{gathered} \tag{4.2.5.5}$$

Substituting this back into the strain equation:

$$\varepsilon = \hat{\boldsymbol{r}}^T \frac{1}{J}\frac{\partial \boldsymbol{N}}{\partial \xi}\boldsymbol{U}^e \tag{4.2.5.6}$$

The unit tangent vector is the Jacobian vector divided by its magnitude:

$$\hat{\boldsymbol{r}}^T = \frac{\boldsymbol{J}^T}{J} = \frac{1}{J}\begin{pmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial y}{\partial \xi} \end{pmatrix} \tag{4.2.5.7}$$

Thus, the strain can be written as:

$$\varepsilon = \frac{\boldsymbol{J}^T}{J^2}\frac{\partial \boldsymbol{N}}{\partial \xi}\boldsymbol{U}^e = \boldsymbol{B}(\xi)\boldsymbol{U}^e \tag{4.2.5.8}$$

Yielding the final expression for the $\boldsymbol{B}$ matrix:

$$\boldsymbol{B}(\xi) = \frac{\boldsymbol{J}^T}{J^2}\frac{\partial \boldsymbol{N}}{\partial \xi} \tag{4.2.5.9}$$

9. Exercise 4.3.4

For the elements below, determine the nodal forces equivalent to body forces. Consider the element area A, thickness t, and unit weight γ.

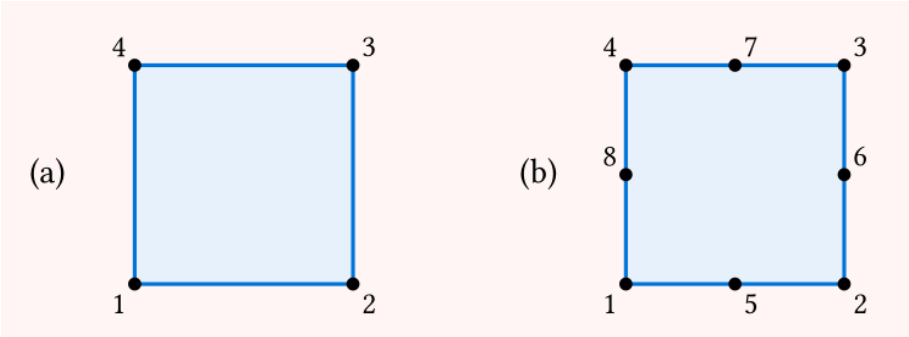

Figure 9.1: 4-node (a) and 8-node (b) Square Elements

9.1 Solution

The equivalent vertical nodal-force vector is

$$\boldsymbol{F}_v = t \int_{A'} \boldsymbol{N}^T b_v J d\eta d\xi \tag{4.3.4.1}$$

Since the Area is A, the Jacobian will be

$$J = \frac{A}{4} \tag{4.3.4.2}$$

9.1.1 (a)

The shape function for a 4-node square element will be

$$N_i = \frac{1}{4}(1 + \xi_i \xi)(1 + \eta_i \eta) \tag{4.3.4.3}$$

Thus

$$\boldsymbol{F}_v = -\frac{\gamma t A}{16} \int_{-1}^{1} \int_{-1}^{1} \begin{pmatrix} (1-\xi)(1-\eta) \\ (1+\xi)(1-\eta) \\ (1+\xi)(1+\eta) \\ (1-\xi)(1+\eta) \end{pmatrix} d\xi d\eta \tag{4.3.4.4}$$

Note that

$$\int_{-1}^{1} (1 \pm x) dx = 2 \tag{4.3.4.5}$$

$$\boldsymbol{F}_v = -\frac{\gamma t A}{16}\begin{pmatrix}4\\4\\4\\4\end{pmatrix} = -\frac{\gamma t A}{4}\begin{pmatrix}1\\1\\1\\1\end{pmatrix} \tag{4.3.4.6}$$

9.1.2 (b)

Similarly, for the 8-node element,

$$\boldsymbol{F}_v = -\frac{\gamma t A}{4}\int_{A'} \boldsymbol{N}^T d\eta d\xi \tag{4.3.4.7}$$

$$\boldsymbol{N}^T = \begin{pmatrix}N_1\\N_2\\N_3\\N_4\\N_5\\N_6\\N_7\\N_8\end{pmatrix} = \begin{pmatrix}\frac{1}{4}(1-\xi)(1-\eta)(-1-\xi-\eta)\\ \frac{1}{4}(1+\xi)(1-\eta)(-1+\xi-\eta)\\ \frac{1}{4}(1+\xi)(1+\eta)(-1+\xi+\eta)\\ \frac{1}{4}(1-\xi)(1+\eta)(-1-\xi+\eta)\\ \frac{1}{2}(1-\eta)(1-\xi^2)\\ \frac{1}{2}(1+\xi)(1-\eta^2)\\ \frac{1}{2}(1+\eta)(1-\xi^2)\\ \frac{1}{2}(1-\xi)(1-\eta^2)\end{pmatrix} \tag{4.3.4.8}$$

Solving it in julia, we obtain

```
using LinearAlgebra

function Nvector(xi, eta)
    return [
        1/4 * (1 - xi) * (1 - eta) * (-1 - xi - eta);
        1/4 * (1 + xi) * (1 - eta) * (-1 + xi - eta);
        1/4 * (1 + xi) * (1 + eta) * (-1 + xi + eta);
        1/4 * (1 - xi) * (1 + eta) * (-1 - xi + eta);
        1/2 * (1 - eta) * (1 - xi ^2);
        1/2 * (1 + xi) * (1 - eta^2);
        1/2 * (1 + eta) * (1 - xi ^2);
        1/2 * (1 - xi) * (1 - eta^2);
    ]
end

quad_2 = [
    (-1 / sqrt(3), 1.0),
    ( 1 / sqrt(3), 1.0)
]
function obtain_integral(Nvector)
    # Initialize result based on the actual length of the vector (8 elements)
    result = zeros(length(Nvector(0.0, 0.0)))



    for i in eachindex(result)
        integral = 0.0
        for (xi, wi) in quad_2
            for (eta, wj) in quad_2
                # Correctly indexing the i-th shape function
                integral += Nvector(xi, eta)[i] * wi * wj
            end
        end
```

```
        result[i] = integral
    end

    return result
end


display(obtain_integral(Nvector))
```

```
8-element Vector{Float64}:
 -0.3333333333333331
 -0.3333333333333331
 -0.3333333333333331
 -0.3333333333333331
  1.333333333333333
  1.333333333333333
  1.333333333333333
  1.3333333333333328
```

$$\int_{A'} \boldsymbol{N}^T d\xi d\eta = [-0.3333 \ -0.3333 \ -0.3333 \ -0.3333 \ 1.3333 \ 1.3333 \ 1.3333 \ 1.3333] \tag{4.3.4.9}$$

Finally

$$\boldsymbol{F}_v = \frac{\gamma t A}{12}[1 \ 1 \ 1 \ 1 \ -4 \ -4 \ -4 \ -4] \tag{4.3.4.10}$$

10. Exercise 4.3.5

Determine the equivalent nodal forces corresponding to the distributed load per unit length q.

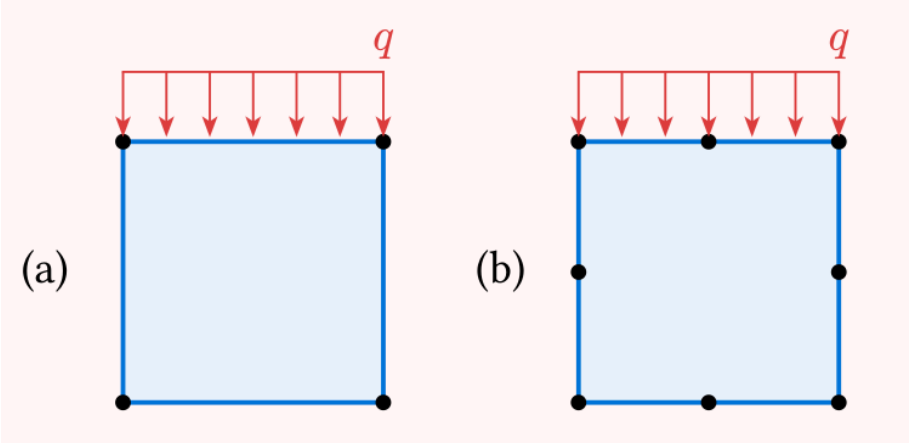

Figure 10.1: distributed vertical load q applied to a 4-node (a) and 8-node (b) square element

10.1 Solution

10.1.1 (a)

For a distributed load q acting on an element edge, the equivalent nodal forces are obtained by integrating the traction over the 1D boundary (the edge) rather than the 2D area:

$$\boldsymbol{F}_t = \int_L \boldsymbol{N}^T \boldsymbol{t} dL = \int_{-1}^{1} \boldsymbol{N}^T \boldsymbol{t} J d\xi \tag{4.3.5.1}$$

The load is a vertical traction per unit length, so the traction vector is simply $\boldsymbol{t} = \begin{pmatrix} 0 \\ -q \end{pmatrix}$.

Since the integration is along a 1D edge of length L, the corresponding line Jacobian is:

$$J = \frac{L}{2} \tag{4.3.5.2}$$

Assuming the load is applied to the top edge connecting nodes 4 and 3, we evaluate the shape functions at $\eta = 1$. The relevant 2D shape functions simplify to 1D linear shape functions along ξ:

$$\begin{aligned} N_4(\xi, 1) &= \frac{1}{2}(1 - \xi) \\ N_3(\xi, 1) &= \frac{1}{2}(1 + \xi) \end{aligned} \tag{4.3.5.3}$$

The vertical nodal forces for these two nodes are evaluated as:

$$\begin{pmatrix} F_{4y} \\ F_{3y} \end{pmatrix} = \int_{-1}^{1} \begin{pmatrix} N_4 \\ N_3 \end{pmatrix} (-q) \left(\frac{L}{2} \right) d\xi \tag{4.3.5.4}$$

$$\begin{pmatrix} F_{4y} \\ F_{3y} \end{pmatrix} = -\frac{qL}{2} \int_{-1}^{1} \begin{pmatrix} \frac{1}{2}(1 - \xi) \\ \frac{1}{2}(1 + \xi) \end{pmatrix} d\xi \tag{4.3.5.5}$$

Because the integral of each 1D linear shape function from -1 to 1 evaluates exactly to 1, we obtain the vertical equivalent forces:

$$\begin{pmatrix} F_{4y} \\ F_{3y} \end{pmatrix} = \frac{qL}{2} \begin{pmatrix} -1 \\ -1 \end{pmatrix} \tag{4.3.5.6}$$

10.1.2 (b)

For the same distributed load q acting now on an 8-node element, the shape functions are that of a 8-node square element, evaluated at $\eta = 1$

$$\boldsymbol{N}^T = \begin{pmatrix} N_3 \\ N_4 \\ N_7 \end{pmatrix} = \begin{pmatrix} \frac{\xi}{2}(\xi+1) \\ \frac{\xi}{2}(\xi-1) \\ 1-\xi^2 \end{pmatrix} \tag{4.3.5.7}$$

$$\begin{pmatrix} F_{4y} \\ F_{7y} \\ F_{3y} \end{pmatrix} = \int_{-1}^{1} \begin{pmatrix} N_4 \\ N_7 \\ N_3 \end{pmatrix} (-q) \left(\frac{L}{2} \right) d\xi = -\frac{qL}{2} \int_{-1}^{1} \begin{pmatrix} \frac{\xi}{2}(\xi+1) \\ \frac{\xi}{2}(\xi-1) \\ 1-\xi^2 \end{pmatrix} d\xi \tag{4.3.5.8}$$

This equates to

$$\begin{pmatrix} F_{4y} \\ F_{7y} \\ F_{3y} \end{pmatrix} = -\frac{qL}{6} \begin{pmatrix} 1 \\ 4 \\ 1 \end{pmatrix} \tag{4.3.5.9}$$

11. Exercise 4.3.6

The nodal coordinates of a 3-node bar element are $(x_1, y_1) = (-1, -1)$, $(x_2, y_2) = (1, 1)$, and $(x_3, y_3) = (0, 1)$. If the unit weight is $\gamma = 70\text{kN/m}^3$ and the area is $A = 0.02\text{m}^2$, calculate the equivalent nodal forces due to the element weight using numerical integration.

11.1 Solution

The following script will solve the problem stated

```
using LinearAlgebra

X = [
    -1.0 -1.0;
     1.0  1.0;
     0.0  1.0
]

γ = 70.0 # kN/m³
A = 0.02 # m²

function body_force()
    return [0.0, -γ * A]
end

function N(xi)
    return [
        0.5 * xi * (xi - 1.0);
        0.5 * xi * (xi + 1.0);
        1.0 - xi^2
    ]
end

function partial_N(xi)
    return [
        xi - 0.5;
        xi + 0.5;
        -2.0 * xi
    ]
end

function Jacobian(X, xi)
    return X' * partial_N(xi)
end

function Fb(N, b, J, X)
    quad_3 = [
        (-sqrt(3/5), 5/9),
        ( 0.0,       8/9),
        ( sqrt(3/5), 5/9)
    ]

    result = zeros(3, 2)
```

```
    for (xi, w) in quad_3
        result += N(xi) * transpose(b()) * norm(J(X, xi)) * w
    end

    return result
end

Fb(N, body_force, Jacobian, X)
```

```
3×2 Matrix{Float64}:
 0.0  -1.38635
 0.0  -0.423948
 0.0  -2.96677
```

12. Exercise 4.3.7

Find the nodal forces corresponding to horizontal and normal tractions with $q = 1$ kN/m as shown below. Assume the element thickness is $h = 1$ m. The coordinates matrix is:

$$\boldsymbol{X} = \begin{pmatrix} 0 & 0 \\ 4 & 0 \\ 4 & 4 \\ 0 & 4 \\ 2 & 0 \\ 3 & 2 \\ 2 & 4 \\ 0 & 2 \end{pmatrix} \text{ m} \tag{4.3.7.1}$$

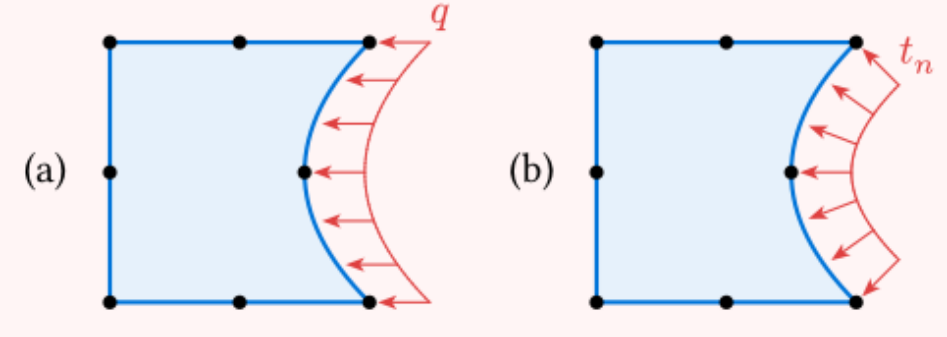

Figure 12.1: 8-node irregular element with horizontal (a) and normal (b) tractions

12.1 Solution

The equivalent nodal forces are

$$\boldsymbol{F}_t = \int_A \boldsymbol{N}^T \boldsymbol{t} J d\xi d\eta \tag{4.3.7.2}$$

The shape functions for the nodes of interest are defined as

$$\begin{pmatrix} N_2 \\ N_3 \\ N_6 \end{pmatrix} = \begin{pmatrix} \frac{\xi}{2}(\xi - 1) \\ \frac{\xi}{2}(\xi + 1) \\ 1 - \xi^2 \end{pmatrix} \tag{4.3.7.3}$$

In the integral being

$$\boldsymbol{N}^T = \begin{pmatrix} N_2 & 0 \\ 0 & N_2 \\ N_3 & 0 \\ 0 & N_3 \\ N_6 & 0 \\ 0 & N_6 \end{pmatrix} \tag{4.3.7.4}$$

The Jacobian depends on the coordinate matrix $\boldsymbol{X}$ and the current parametric coordinate ξ for the shape function:

$$J(\xi) = \boldsymbol{X}^T \frac{d\boldsymbol{N}(\xi)}{d\boldsymbol{\xi}} \tag{4.3.7.5}$$

12.1.1 (a)

The traction vector is then defined as

$$\boldsymbol{t} = \begin{pmatrix} -q \\ 0 \end{pmatrix} \tag{4.3.7.6}$$

Then we must define

$$\boldsymbol{F}_t = \int_{-1}^{1} \begin{pmatrix} N_2 & 0 \\ 0 & N_2 \\ N_3 & 0 \\ 0 & N_3 \\ N_6 & 0 \\ 0 & N_6 \end{pmatrix} \begin{pmatrix} -q \\ 0 \end{pmatrix} ||J|| d\xi \tag{4.3.7.7}$$

For the x coordinate, equates to

$$F_{xt} = -q \int_{-1}^{1} \begin{pmatrix} N_2 \\ N_3 \\ N_6 \end{pmatrix} ||J|| d\xi \tag{4.3.7.8}$$

To solve this in julia:

```
using LinearAlgebra


function main()

    X = [
        4 0;
        4 4;
        3 2;
    ]

    dNdxi(xi) = [
        xi - 1/2;
        xi + 1/2;
        -2 * xi
    ]


    # define quad_2 values
    quad_2 = [
        (-1 / sqrt(3), 1.0),
        ( 1 / sqrt(3), 1.0)
    ]

    # shape functions as vector
    N(xi) = [
        xi / 2 * (xi - 1);
        xi / 2 * (xi + 1);
        1 - xi^2
```

```
    ]

    result = zeros(3)

    for (xi, w) in quad_2
        result += N(xi) * norm(X' * dNdxi(xi)) * w
    end

    return result
end

main()
```

```
3-element Vector{Float64}:
 0.7698003589195012
 0.7698003589195008
 3.0792014356780024
```

Thus

$$F_{xt} = \begin{pmatrix} F_{2xt} \\ F_{3xt} \\ F_{6xt} \end{pmatrix} = -q \begin{bmatrix} 0.7698 \\ 0.7698 \\ 3.0792 \end{bmatrix} \tag{4.3.7.9}$$

12.1.2 (b)

In relation to the last problem, the only difference is defined in the tension vector, which is now:

$$\boldsymbol{t}(\xi) = -\hat{n}(\xi)q \tag{4.3.7.10}$$

We can define the normal vector with the Jacobian:

$$\hat{n}(\xi) \perp \hat{J}(\xi) = \frac{\boldsymbol{J}}{\|J\|} \tag{4.3.7.11}$$

For a 2D Jacobian vector, then:

$$\hat{n} = \begin{pmatrix} \hat{J}_2 & -\hat{J}_1 \end{pmatrix}^T \tag{4.3.7.12}$$

So, simply:

$$\boldsymbol{F}_t = -q \int_{-1}^{1} \begin{pmatrix} N_2 & 0 \\ 0 & N_2 \\ N_3 & 0 \\ 0 & N_3 \\ N_6 & 0 \\ 0 & N_6 \end{pmatrix} \begin{pmatrix} \hat{n}_x \\ \hat{n}_y \end{pmatrix} \|J\| d\xi \tag{4.3.7.13}$$

```
using LinearAlgebra


function main()

    X = [
        4 0;
        4 4;
        3 2;
    ]
```

```
    dNdxi(xi) = [
        xi - 1/2;
        xi + 1/2;
        -2 * xi
    ]


    # define quad_2 values
    quad_2 = [
        (-1 / sqrt(3), 1.0),
        ( 1 / sqrt(3), 1.0)
    ]

    # shape functions as vector
    N(xi) = [
        (xi/2*(xi-1)) 0;
        0 (xi/2*(xi-1));
        (xi / 2 * (xi + 1)) 0;
        0 (xi / 2 * (xi + 1));
        (1 - xi^2) 0;
        0 (1 - xi^2);
    ]



    function normal(xi)
        J = X' * dNdxi(xi)
        hatJ = J / norm(J)
        return [
            hatJ[2];
            -hatJ[1];
        ]
    end

    result = zeros(6)

    for (xi, w) in quad_2
        result += N(xi) * normal(xi) * norm(X' * dNdxi(xi)) * w
    end

    return result
end

main()
```

```
6-element Vector{Float64}:
  0.6666666666666667
  0.6666666666666662
  0.6666666666666665
 -0.6666666666666662
  2.666666666666665
  0.0
```

Thus

$$\boldsymbol{F}_t = \begin{pmatrix} F_{2tx} \\ F_{2ty} \\ F_{3tx} \\ F_{3ty} \\ F_{6tx} \\ F_{6ty} \end{pmatrix} = \frac{-q}{3} \begin{bmatrix} 2 \\ 2 \\ 2 \\ -2 \\ 8 \\ 0 \end{bmatrix} \tag{4.3.7.14}$$

13. Exercise 4.3.8

For the element below, determine the corresponding nodal forces if a normal traction $t_n = 1$ kPa is applied on the shaded face.

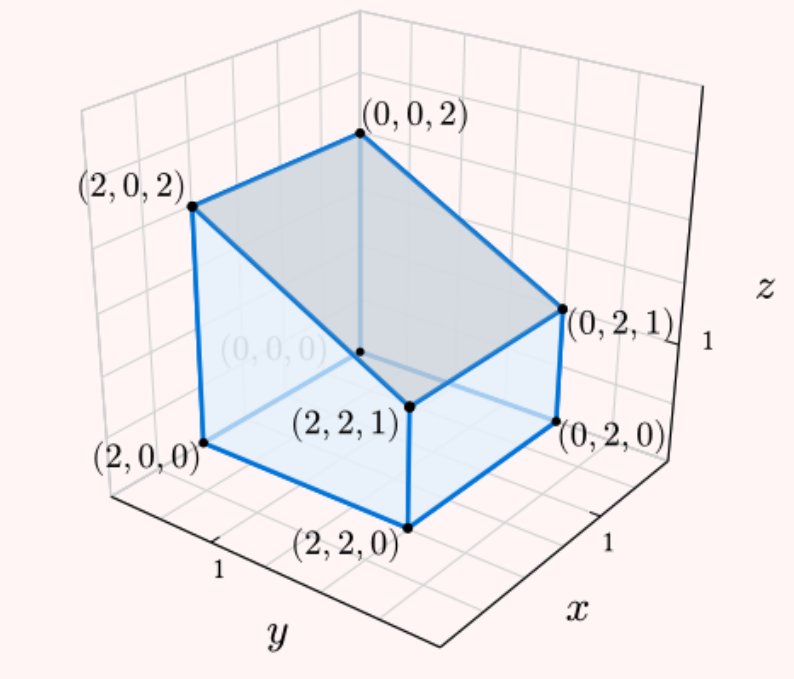

Figure 13.1: Element provided

13.1 Solution

Normal traction is defined as

$$\boldsymbol{F}_n = t_n \int_A \boldsymbol{N}^T \hat{\boldsymbol{n}} dA \tag{4.3.8.1}$$

The shape functions for a 4-node square 3D plane are

$$N_i = \frac{1}{4}(1 + \xi_i \xi)(1 + \eta_i \eta) \tag{4.3.8.2}$$

Thus, to solve this problem, we will use the following julia script

```
using LinearAlgebra


function main()
    tn = -1 # kPa

    X = [
        2 2 1;
        0 2 1;
```

```
        0 0 2;
        2 0 2;
    ]

    # shape functions as vector
    N(xi, eta) = [
        1/4 * (1 - xi) * (1 - eta);
        1/4 * (1 + xi) * (1 - eta);
        1/4 * (1 + xi) * (1 + eta);
        1/4 * (1 - xi) * (1 + eta);
    ]
    # its derivatives
    dNdxi(xi, eta) = [
        (1/4 * (-1 + eta))   (1/4 * (-1 + xi));
        (1/4 * ( 1 - eta))   (1/4 * (-1 - xi));
        (1/4 * ( 1 + eta))   (1/4 * ( 1 + xi));
        (1/4 * (-1 - eta))   (1/4 * ( 1 - xi));
    ]

    N_matrix(xi, eta) = transpose([
        N(xi, eta)[1] * Matrix(1I, 3,3)
        N(xi, eta)[2] * Matrix(1I, 3,3)
        N(xi, eta)[3] * Matrix(1I, 3,3)
        N(xi, eta)[4] * Matrix(1I, 3,3)
    ])

    Jacobian(xi, eta) = X' * dNdxi(xi, eta)

    function normal(xi, eta)
        J = Jacobian(xi, eta)
        n = cross(J[:, 1], J[:, 2])
        return n / norm(n)
    end

    quad_3 = [
        (-sqrt(3/5), 5/9),
        (0, 8/9),
        (sqrt(3/5), 5/9),
    ]

    # solving the integral
    result = zeros(12)

    for (xi, wi) in quad_3
        for (eta, wj) in quad_3
            J = Jacobian(xi, eta)
             result += N_matrix(xi, eta)' * normal(xi, eta) * sqrt(det(J' * J)) * wi
* wj
        end
    end

    return result * tn
end

main()
```

```
12-element Vector{Float64}:
 -0.0
 -0.5
 -1.0
```

```
-0.0
-0.5
-1.0
-0.0
-0.5
-1.0
-0.0
-0.5
-1.0
```

Thus, according to the local mapping (node 1 at $(2, 2, 1)$)

$$\begin{pmatrix} F_{x_1} \\ F_{y_1} \\ F_{z_1} \\ F_{x_2} \\ F_{y_2} \\ F_{z_2} \\ F_{x_3} \\ F_{y_3} \\ F_{z_3} \\ F_{x_4} \\ F_{y_4} \\ F_{z_4} \end{pmatrix} = - \begin{pmatrix} 0.0 \\ 0.5 \\ 1.0 \\ 0.0 \\ 0.5 \\ 1.0 \\ 0.0 \\ 0.5 \\ 1.0 \\ 0.0 \\ 0.5 \\ 1.0 \end{pmatrix} \tag{4.3.8.3}$$

14. Exercise 4.4.5

Using computer software, compute the stiffness matrix for the triangular element. Use $E = 20$ GPa and $\nu = 0.25$.

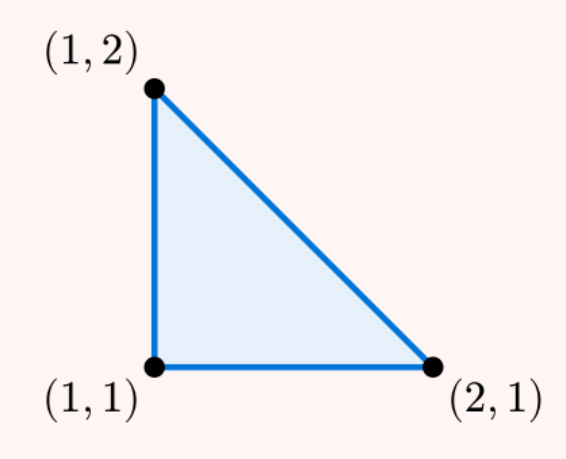

Figure 14.1: Triangular element with coordinates $(1, 1)$, $(2, 1)$, and $(1, 2)$

14.1 Solution

For a 3-node triangular element, the strain-displacement matrix $\boldsymbol{B}$ and the constitutive matrix $\boldsymbol{D}$ are strictly constant over the entire element domain, which simplifies the stiffness matrix:

$$\boldsymbol{K} = \int_A \boldsymbol{B}^T \boldsymbol{D} \boldsymbol{B} h dA = \boldsymbol{B}^T \boldsymbol{D} \boldsymbol{B} h A \tag{4.4.5.1}$$

Assuming a plane stress condition and a unit thickness ($h = 1$), the coordinate matrix for the given element is:

$$\boldsymbol{C} = \begin{pmatrix} 1 & 1 \\ 2 & 1 \\ 1 & 2 \end{pmatrix} \text{ m} \tag{4.4.5.2}$$

The derivatives of the linear shape functions with respect to the natural coordinates (ξ, η) are constant for all integration points:

$$\frac{\partial \boldsymbol{N}}{\partial \boldsymbol{\xi}} = \begin{pmatrix} -1 & -1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix} \tag{4.4.5.3}$$

The Jacobian matrix evaluates perfectly to the identity matrix, meaning the element is an undistorted right triangle:

$$\boldsymbol{J} = \boldsymbol{C}^T \frac{\partial \boldsymbol{N}}{\partial \boldsymbol{\xi}} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \tag{4.4.5.4}$$

Since $\det(\boldsymbol{J}) = 1$, the area of the element is $A = \frac{1}{2} \det(\boldsymbol{J}) = 0.5 \text{m}^2$. We can define the inverse of the Jacobian as

$$\boldsymbol{J}^{-1} = \frac{\text{adj}(\boldsymbol{J})}{\det(\boldsymbol{J})} = \boldsymbol{I} \tag{4.4.5.5}$$

Thus,

$$\boldsymbol{B} = \begin{pmatrix} \frac{\partial N_1}{\partial x} & 0 & \frac{\partial N_2}{\partial x} & 0 & \frac{\partial N_3}{\partial x} & 0 \\ 0 & \frac{\partial N_1}{\partial y} & 0 & \frac{\partial N_2}{\partial y} & 0 & \frac{\partial N_3}{\partial y} \\ \frac{\partial N_1}{\partial y} & \frac{\partial N_1}{\partial x} & \frac{\partial N_2}{\partial y} & \frac{\partial N_2}{\partial x} & \frac{\partial N_3}{\partial y} & \frac{\partial N_3}{\partial x} \end{pmatrix} = \begin{pmatrix} -1 & 0 & 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 & 1 \\ -1 & -1 & 0 & 1 & 1 & 0 \end{pmatrix} \tag{4.4.5.6}$$

Finally, substituting $E = 20$ GPa and $\nu = 0.25$ into the plane stress constitutive matrix $\boldsymbol{D}$, the constant stiffness matrix $\boldsymbol{K}$ can be evaluated analytically or numerically.

```
using LinearAlgebra

function main()
    # coords
    C = [
        1.0  1.0;
        2.0  1.0;
        1.0  2.0
    ]

    # Material properties and geometry
    E = 20e9 # Pa
    nu = 0.25
    h = 1.0 # assumed unit thicness

    # matrix D
    D = (E / (1 - nu^2)) * [
        1.0   nu    0.0;
        nu    1.0   0.0;
        0.0   0.0   (1 - nu)/2
    ]

    dNdxi = [
        -1.0 -1.0;
         1.0  0.0;
         0.0  1.0
    ]

    # Jacobian, in this case constant
    J = C' * dNdxi

    # Calculate the area
    Area = 0.5 * abs(det(J))

    # Cartesian derivatives of the shape functions
    dNdX = dNdxi * inv(J)

    # Assemble the B matrix
    B = zeros(3, 6)
    for i in 1:3
        col = (i - 1) * 2
        B[1, col + 1] = dNdX[i, 1]
        B[2, col + 2] = dNdX[i, 2]
        B[3, col + 1] = dNdX[i, 2]
        B[3, col + 2] = dNdX[i, 1]
    end
```

```
    # Compute the stiffness matrix (B and D are constant, so numerical integration loop
is bypassed)
    K = B' * D * B * h * Area

    return K
end


display(main())
```

```
6×6 Matrix{Float64}:
  1.46667e10   6.66667e9   -1.06667e10  -4.0e9  -4.0e9  -2.66667e9
  6.66667e9    1.46667e10  -2.66667e9   -4.0e9  -4.0e9  -1.06667e10
 -1.06667e10  -2.66667e9    1.06667e10   0.0     0.0     2.66667e9
 -4.0e9       -4.0e9        0.0          4.0e9   4.0e9   0.0
 -4.0e9       -4.0e9        0.0          4.0e9   4.0e9   0.0
 -2.66667e9   -1.06667e10   2.66667e9    0.0     0.0     1.06667e10
```

Thus

$$\boldsymbol{K} = \begin{bmatrix} 14.6667 & 6.6667 & -10.6667 & -4 & -4 & -2.6667 \\ 6.6667 & 14.6667 & -2.6667 & -4 & -4 & -10.6667 \\ -10.6667 & -2.6667 & 10.6667 & 0 & 0 & 2.6667 \\ -4 & -4 & 0 & 4 & 4 & 0 \\ -4 & -4 & 0 & 4 & 4 & 0 \\ -2.6667 & -10.6667 & 2.6667 & 0 & 0 & 10.6667 \end{bmatrix} \cdot 10^9 \frac{N}{m} \tag{4.4.5.7}$$

15. Exercise 4.4.6

The nodal coordinates of a three-node bar element are

$$\boldsymbol{C} = \begin{pmatrix} -1 & -1 \\ 1 & 1 \\ 0 & 1 \end{pmatrix} \text{ m} \tag{4.4.6.1}$$

At the end of a finite-element analysis, the axial stresses at the three integration points are $\sigma_1 = 5$ kPa, $\sigma_2 = 10$ kPa, and $\sigma_3 = 15$ kPa. Compute the nodal internal forces vector.

15.1 Solution

As defined by the following script:

```
using LinearAlgebra

function main()
    σ = [
        5,
        10,
        15
    ] * 1e3 # Pa

    A = 1 #m²

    C = [
        -1 -1;
         1  1;
         0  1;
    ] # m

    N(xi) = [
        0.5 * xi * (xi - 1);
        0.5 * xi * (xi + 1);
        1 - xi^2
    ]

    function N_2d_matrix(xi)
        m = nothing
        for i in 1:length(N(xi))
            subN = dNdxi(xi)[i] * Matrix(1I, 2, 2)
            if m === nothing
                m = subN
            else
                m = [m subN]
            end
        end
        return m
    end

    dNdxi(xi) = [
        xi - 0.5;
        xi + 0.5;
        -2 * xi;
```

```
    ]

    Jacobian(xi) = C' * dNdxi(xi)

    function B(xi)
        J = Jacobian(xi)
        return J' / (norm(J)^2) * N_2d_matrix(xi)
    end

    quad_3 = [
        (-sqrt(3/5), 5/9),
        (0, 8/9),
        (sqrt(3/5), 5/9),
    ]

    result = zeros(6)
    for (i, (xi, w)) in enumerate(quad_3)
        result += B(xi)' * σ[i] * norm(Jacobian(xi)) * w
    end


    return result * A

end

main() # N
```

```
6-element Vector{Float64}:
 -2429.928026819304
 -7540.24644964248
 12174.161585552662
 -2680.390396003872
 -9744.23355873336
 10220.636845646353
```

Thus

$$\boldsymbol{F} = \begin{bmatrix} -2.4299 \\ -7.5402 \\ 12.1742 \\ -2.6804 \\ -9.7442 \\ 10.2206 \end{bmatrix} \cdot kN \tag{4.4.6.2}$$

16. Exercise 4.4.7

The nodal coordinates of a four-node quadrilateral element are

$$\boldsymbol{C} = \begin{pmatrix} 0 & 0 \\ 2 & 0 \\ 2 & 2 \\ 0 & 2 \end{pmatrix} \text{ m} \tag{4.4.7.1}$$

and the nodal displacements are

$$\boldsymbol{U}^{(e)} = (0.0, 0.0, 0.001, 0.0, 0.002, -0.001, -0.001, -0.001) \text{ m} \tag{4.4.7.2}$$

Assuming full integration, compute the internal nodal-force vector. Use $E = 20$ GPa and $\nu = 0.1$.

16.1 Solution

The following script solves for the element above, and any other similar element:

```
using LinearAlgebra

function getK(C, E, ν, h)


    # for plane stress
    D = [
        1 ν 0;
        ν 1 0;
        0 0 (1 - ν) / 2;
    ] * E / (1 - ν ^ 2)

    function Ni(xi_i, eta_i)
        return N(xi, eta) = 0.25 * (1 + xi_i * xi) * (1 + eta_i * eta)
    end

    function dNi(xi_i, eta_i)
        return dN(xi, eta) = [
            ((xi_i + xi_i * eta_i * eta) / 4) ((eta_i + xi_i * eta_i * xi )/ 4);
        ]
    end

    N(xi, eta) = [
        Ni(-1, -1)(xi, eta);
        Ni( 1, -1)(xi, eta);
        Ni( 1,  1)(xi, eta);
        Ni(-1,  1)(xi, eta);
    ]

    dNdxi(xi, eta) = [
        dNi(-1, -1)(xi, eta);
        dNi( 1, -1)(xi, eta);
        dNi( 1,  1)(xi, eta);
        dNi(-1,  1)(xi, eta);
    ]
```

```
    Jacobian(xi, eta) = C' * dNdxi(xi, eta)


    function B(xi, eta)
        J = Jacobian(xi, eta)
        dN_dξη = dNdxi(xi, eta)

        # transform derivatives
        dN_dxdy = dN_dξη * inv(J)

        Bmat = zeros(3, 8)

        for (i, (dx, dy)) in enumerate(eachrow(dN_dxdy))
            Bmat[:, 2i-1:2i] .= [
                dx 0
                0  dy
                dy dx
            ]
        end

        return Bmat
    end

    quad_2 = [
        (-1 / sqrt(3), 1.0),
        ( 1 / sqrt(3), 1.0)
    ]

    result = zeros(8,8)

    # final integration
    for (xi, wi) in quad_2
        for (eta, wj) in quad_2
            result += h * B(xi, eta)' * D * B(xi, eta) * det(Jacobian(xi, eta)) * wi
* wj
        end
    end

    return result
end

# geometry and material definitions
C = [
    0 0;
    2 0;
    2 2;
    0 2;
]
# thickness
h = 1 #m

E = 20e9 # Pa
ν = .1

# displacements
U = [
    0.0 0.0 0.001 0.0 0.002 -0.001 -0.001 -0.001
]' #m

k = getK(C, E, ν, h)
```

```
F = k * U
```

```
8×1 Matrix{Float64}:
 -1.4309764309764307e7
  8.080808080808082e6
  1.4309764309764307e7
  8.080808080808081e6
  2.4074074074074075e7
 -8.080808080808083e6
 -2.4074074074074075e7
 -8.080808080808081e6
```

Thus

$$\boldsymbol{F} = \begin{bmatrix} -14.3098 \\ 8.0808 \\ 14.3098 \\ 8.0808 \\ 24.0741 \\ -8.0808 \\ -24.0741 \\ -8.0808 \end{bmatrix} \text{ MN} \tag{4.4.7.3}$$

17. Exercise 4.4.8

The two-element mesh was designed to simulate self-weight in a structure under plane stress conditions. Each element has dimensions 1×1 m. Considering full integration, compute:

a. The stiffness matrix for each element.
b. The equivalent nodal loads corresponding to the body force.
c. The global stiffness matrix and the global force vector.
d. The nodal displacements and reactions.
e. The strain and stress vectors at the integration points of element 1.
f. Check if the vertical stress at a lower integration point of element 1 is close to the analytical value γh.

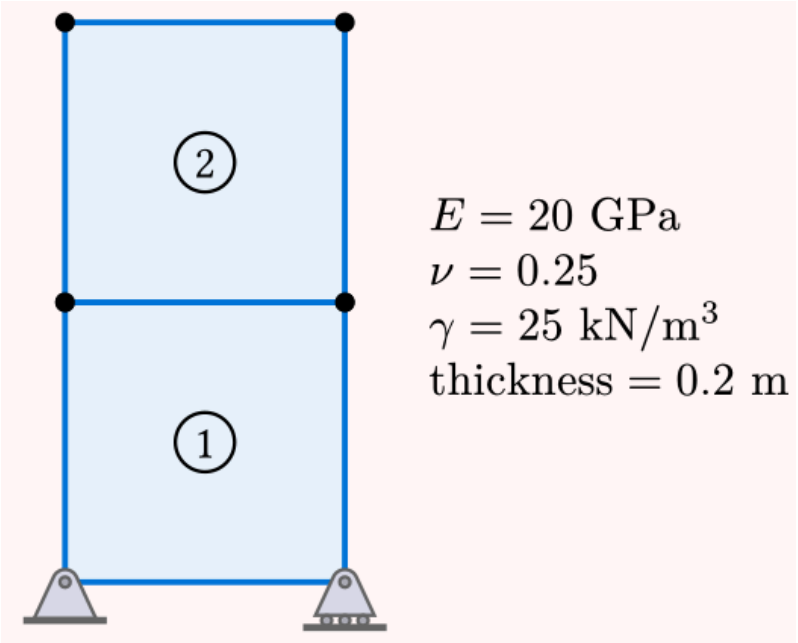

Figure 17.1: Two-element mesh simulating self-weight with $E = 20$ GPa, $\nu = 0.25$, $\gamma = 25\text{kN/m}^3$, and thickness $= 0.2$ m

17.1 Solution

This solution aims to be a review of all ideas presented in the chapters preceding it. Thus, for each cell of julia code, a small definition of all elements will be defined.

17.1.1 (a)

First step if to define the elements's stiffness matrix. Well, from the definition of virtual work, which, in simpler terms, relates the effects of deformations and virtual displacements:

$$\int_{\boldsymbol{\Omega}^{(e)}} \delta\boldsymbol{\varepsilon} \cdot \boldsymbol{\sigma} \, \mathrm{dV} = \int_{\boldsymbol{\Omega}^{(e)}} \delta\boldsymbol{u} \cdot \boldsymbol{b} \, \mathrm{dV} + \int_{\boldsymbol{\Gamma}_t^{(e)}} \delta\boldsymbol{u} \cdot \boldsymbol{t} \, \mathrm{dS} \tag{4.4.8.1}$$

Both strains and virtual displacements can be defined relating to the nodal displacements $\delta\boldsymbol{U}$

$$\delta\boldsymbol{u} = \boldsymbol{N}\delta\boldsymbol{U}^{(e)} \tag{4.4.8.2}$$

$$\delta\boldsymbol{\xi} = \boldsymbol{B}\delta\boldsymbol{U}^{(e)} \tag{4.4.8.3}$$

Which can simplify (4.4.8.1) into:

$$\int_{\boldsymbol{\Omega}^{(e)}} \boldsymbol{B}^T\boldsymbol{\sigma}\ \mathrm{dV} = \int_{\boldsymbol{\Omega}^{(e)}} \boldsymbol{N}^T\boldsymbol{b}\ \mathrm{dV} + \int_{\boldsymbol{\Gamma}_t^{(e)}} \boldsymbol{N}^T\boldsymbol{t}\ \mathrm{dS} \tag{4.4.8.4}$$

As I see it, (4.4.8.4) relates the internal tensions in the body with forces applied to it, let it be internal (like body mass or eletromagnetic forces) or external (standard forces, water pressure, etc.). So, our stiffness matrix is defined in the left side of the equation (4.4.8.4).

For **small-strain linear elasticity** we have:

$$\boldsymbol{\sigma} = \boldsymbol{DBU}^{(e)} \tag{4.4.8.5}$$

Hence,

$$\int_{\boldsymbol{\Omega}^{(e)}} \boldsymbol{B}^T\boldsymbol{\sigma}\ \mathrm{dV} = \left(\int_{\boldsymbol{\Omega}^{(e)}} \boldsymbol{B}^T\boldsymbol{DB}\ \mathrm{dV}\right)\boldsymbol{U}^{(e)} = \boldsymbol{K}^{(e)}\boldsymbol{U}^{(e)} \tag{4.4.8.6}$$

The following script will then, as defined by the last problem, calculate $\boldsymbol{K}$ with the integral defined above:

```
using LinearAlgebra

# Since all elements have the same materials, firstly we define them

E = 20 * 1e9 # GPa
ν = .25
γ = 25 * 1e3 # kN/m³
h = .2 #m


function Ni(xi_i, eta_i)
    return N(xi, eta) = 0.25 * (1 + xi_i * xi) * (1 + eta_i * eta)
end

function dNi(xi_i, eta_i)
    return dN(xi, eta) = [
        ((xi_i + xi_i * eta_i * eta) / 4) ((eta_i + xi_i * eta_i * xi )/ 4);
    ]
end

N(xi, eta) = [
    Ni(-1, -1)(xi, eta);
    Ni( 1, -1)(xi, eta);
    Ni( 1,  1)(xi, eta);
    Ni(-1,  1)(xi, eta);
]

dNdxi(xi, eta) = [
    dNi(-1, -1)(xi, eta);
    dNi( 1, -1)(xi, eta);
    dNi( 1,  1)(xi, eta);
    dNi(-1,  1)(xi, eta);
]

Jacobian(C, xi, eta) = C' * dNdxi(xi, eta)



function get_B(C, xi, eta)
J = Jacobian(C, xi, eta)
```

```
dN_dξη = dNdxi(xi, eta)

# transform derivatives
dN_dxdy = dN_dξη * inv(J)

Bmat = zeros(3, 8)

for (i, (dx, dy)) in enumerate(eachrow(dN_dxdy))
    Bmat[:, 2i-1:2i] .= [
        dx 0
        0  dy
        dy dx
    ]
end

return Bmat
end


# for plane stress (2D problem, with no stress in the z direction)
get_D(E, ν) = [
    1 ν 0;
    ν 1 0;
    0 0 (1 - ν) / 2;
] * E / (1 - ν ^ 2)


quad_2 = [
    (-1 / sqrt(3), 1.0),
    ( 1 / sqrt(3), 1.0)
]

function get_k(C, D, h)

    quad_2 = [
        (-1 / sqrt(3), 1.0),
        ( 1 / sqrt(3), 1.0)
    ]

    result = zeros(8,8)

    # final integration
    for (xi, wi) in quad_2
        for (eta, wj) in quad_2
            B = get_B(C, xi, eta)
            result += h * B' * D * B * det(Jacobian(C, xi, eta)) * wi * wj
        end
    end

    return result
end

C1 = [
    0 0;
    1 0;
    1 1;
    0 1;
]

C2 = [
```

```
    0 1;
    1 1;
    1 2;
    0 2;
]

k1 = get_k(C1, get_D(E, ν), h)
k2 = get_k(C2, get_D(E, ν), h)
```

```
8×8 Matrix{Float64}:
  1.95556e9   6.66667e8  -1.15556e9  …  -6.66667e8   1.77778e8   1.33333e8
  6.66667e8   1.95556e9   1.33333e8     -9.77778e8  -1.33333e8  -1.15556e9
 -1.15556e9   1.33333e8   1.95556e9     -1.33333e8  -9.77778e8   6.66667e8
 -1.33333e8   1.77778e8  -6.66667e8     -1.15556e9   6.66667e8  -9.77778e8
 -9.77778e8  -6.66667e8   1.77778e8      6.66667e8  -1.15556e9  -1.33333e8
 -6.66667e8  -9.77778e8  -1.33333e8  …   1.95556e9   1.33333e8   1.77778e8
  1.77778e8  -1.33333e8  -9.77778e8      1.33333e8   1.95556e9  -6.66667e8
  1.33333e8  -1.15556e9   6.66667e8      1.77778e8  -6.66667e8   1.95556e9
```

Thus,

$$k_1^{(e)} = k_2^{(e)} = \begin{bmatrix} 1.9556 & 0.6667 & -1.1556 & -0.1333 & -0.9778 & -0.6667 & 0.1778 & 0.1333 \\ 0.6667 & 1.9556 & 0.1333 & 0.1778 & -0.6667 & -0.9778 & -0.1333 & -1.1556 \\ -1.1556 & 0.1333 & 1.9556 & -0.6667 & 0.1778 & -0.1333 & -0.9778 & 0.6667 \\ -0.1333 & 0.1778 & -0.6667 & 1.9556 & 0.1333 & -1.1556 & 0.6667 & -0.9778 \\ -0.9778 & -0.6667 & 0.1778 & 0.1333 & 1.9556 & 0.6667 & -1.1556 & -0.1333 \\ -0.6667 & -0.9778 & -0.1333 & -1.1556 & 0.6667 & 1.9556 & 0.1333 & 0.1778 \\ 0.1778 & -0.1333 & -0.9778 & 0.6667 & -1.1556 & 0.1333 & 1.9556 & -0.6667 \\ 0.1333 & -1.1556 & 0.6667 & -0.9778 & -0.1333 & 0.1778 & -0.6667 & 1.9556 \end{bmatrix} 10^{-9} \quad (4.4.8.7)$$

17.1.2 (b)

Next, we determine the equivalent nodal loads corresponding to the body force (self-weight). From the right side of the virtual work equation (4.4.8.4), the external virtual work due to body forces is:

$$\delta W_{\text{ext},b} = \int_{\boldsymbol{\Omega}^{(e)}} \delta \boldsymbol{u} \cdot \boldsymbol{b} \, \mathrm{dV} \quad (4.4.8.8)$$

Substituting the virtual displacement interpolation $\delta \boldsymbol{u} = \boldsymbol{N} \delta \boldsymbol{U}^{(e)}$, we get:

$$\delta W_{\text{ext},b} = \left(\delta \boldsymbol{U}^{(e)}\right)^T \int_{\boldsymbol{\Omega}^{(e)}} \boldsymbol{N}^T \boldsymbol{b} \, \mathrm{dV} \quad (4.4.8.9)$$

Equating this to the work done by the discrete equivalent nodal forces, $\left(\delta \boldsymbol{U}^{(e)}\right)^T \boldsymbol{F}_b^{(e)}$, we obtain the fundamental expression for the element body force vector:

$$\boldsymbol{F}_b^{(e)} = \int_{\boldsymbol{\Omega}^{(e)}} \boldsymbol{N}^T \boldsymbol{b} \, \mathrm{dV} \quad (4.4.8.10)$$

For a 2D plane stress element with thickness h, mapping the volume differential to the natural coordinates (ξ, η) yields:

$$\boldsymbol{F}_b^{(e)} = h \int_{-1}^{1} \int_{-1}^{1} \boldsymbol{N}^T \boldsymbol{b} J \, \mathrm{d}\xi \, \mathrm{d}\eta \quad (4.4.8.11)$$

Since the structure is subjected to self-weight, the body force vector acts purely in the vertical direction, meaning $\boldsymbol{b} = \begin{pmatrix} 0 \\ -\gamma \end{pmatrix}$. The following script will evaluate this integral numerically to compute the equivalent load vector for each element.

```
function body_force(C, γ, h)
    N_matrix(xi, eta) = hcat(
        [N(xi, eta)[i]*Matrix(1I, 2, 2) for i in 1:4]...
    )

    # assuming b will always be applied by gravity
    b = γ * [0;-1]

    # functions are bilinear (2x2)



    result = zeros(2*4)
    for (xi, wi) in quad_2
        for (eta, wj) in quad_2
            result += N_matrix(xi, eta)' * b * det(Jacobian(C, xi, eta)) * wi * wj
        end
    end
    return result * h
end

bf1 = body_force(C1, γ, h)
bf2 = body_force(C2, γ, h)

println([bf1 bf2])
```

```
[0.0 0.0; -1250.0 -1249.9999999999998; … ; 0.0 0.0; -1250.0 -1249.9999999999998]
```

17.1.3 (c)

After obtaining the individual element stiffness matrices and equivalent load vectors, the next step is to combine them into a single global algebraic system that represents the entire structure. This global system of equations is given by:

$$\boldsymbol{K}\boldsymbol{U} = \boldsymbol{F} \tag{4.4.8.12}$$

The assembly process builds this system by enforcing compatibility between adjacent elements through their shared nodal degrees of freedom. Each element's stiffness matrix $\boldsymbol{K}^{(e)}$ and load vector $\boldsymbol{F}^{(e)}$ are accumulated into the global stiffness matrix $\boldsymbol{K}$ and the global force vector $\boldsymbol{F}$.

The exact placement of these element contributions into the global system is determined by the element's location vector $\ell^{(e)}$, which maps the element's local degrees of freedom to the correct rows and columns in the global arrays.

The element will be defined according to the following matrix, that relates the node coordinates with their DOF:

$$\begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 1 \\ 0 & 1 \\ 1 & 2 \\ 0 & 2 \end{pmatrix} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \\ 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{pmatrix} = \begin{pmatrix} i & i+1 \\ \vdots & \vdots \end{pmatrix} \tag{4.4.8.13}$$

```
global_index_mapper = [
    Dict(
    "coords"=> C1,
    "nodes"=>(1,2,3,4),
```

```
    "k"=>k1,
    "f"=>bf1
    ),
    Dict(
    "coords"=> C2,
    "nodes"=>(4,3,5,6),
    "k"=>k2,
    "f"=>bf2
    )
]



function assemble_Stiffness(gim)
    # get dimensions of global stiffness
    num_nodes = length(Set(n for d in global_index_mapper for n in d["nodes"]))

    K = zeros(2num_nodes, 2num_nodes)
    F = zeros(2num_nodes)

    for element in gim
        k, f = element["k"], element["f"]
        nodes = element["nodes"]

        dofs = Int[]

        for n in nodes
            push!(dofs, 2n - 1) # ux
            push!(dofs, 2n )    # uy
        end

        for i in 1:8
            gI = dofs[i]

            F[gI] += f[i]

            for j in 1:8
                gJ = dofs[j]
                K[gI, gJ] += k[i, j]
            end
        end

    end

    return K, F
end

K, F = assemble_Stiffness(global_index_mapper)
```

```
([1.9555555555555553e9   6.666666666666666e8   …   0.0   0.0;   6.666666666666666e8
1.955555555555555e9 … 0.0 0.0; … ; 0.0 0.0 … 1.9555555555555553e9 -6.666666666666667e8;
0.0 0.0 … -6.666666666666667e8 1.955555555555556e9], [0.0, -1250.0, 0.0, -1250.0, 0.0,
-2500.0, 0.0, -2500.0, 0.0, -1249.9999999999998, 0.0, -1249.9999999999998])
```

Thus:

$$K = \begin{bmatrix} 1.96 & 0.67 & -1.16 & -0.13 & -0.98 & -0.67 & 0.18 & 0.13 & 0 & 0 & 0 & 0 \\ 0.67 & 1.96 & 0.13 & 0.18 & -0.67 & -0.98 & -0.13 & -1.16 & 0 & 0 & 0 & 0 \\ -1.16 & 0.13 & 1.96 & -0.67 & 0.18 & -0.13 & -0.98 & 0.67 & 0 & 0 & 0 & 0 \\ -0.13 & 0.18 & -0.67 & 1.96 & 0.13 & -1.16 & 0.67 & -0.98 & 0 & 0 & 0 & 0 \\ -0.98 & -0.67 & 0.18 & 0.13 & 3.91 & 0 & -2.31 & 0 & 0.18 & -0.13 & -0.98 & 0.67 \\ -0.67 & -0.98 & -0.13 & -1.16 & 0 & 3.91 & 0 & 0.36 & 0.13 & -1.16 & 0.67 & -0.98 \\ 0.18 & -0.13 & -0.98 & 0.67 & -2.31 & 0 & 3.91 & 0 & -0.98 & -0.67 & 0.18 & 0.13 \\ 0.13 & -1.16 & 0.67 & -0.98 & 0 & 0.36 & 0 & 3.91 & -0.67 & -0.98 & -0.13 & -1.16 \\ 0 & 0 & 0 & 0 & 0.18 & 0.13 & -0.98 & -0.67 & 1.96 & 0.67 & -1.16 & -0.13 \\ 0 & 0 & 0 & 0 & -0.13 & -1.16 & -0.67 & -0.98 & 0.67 & 1.96 & 0.13 & 0.18 \\ 0 & 0 & 0 & 0 & -0.98 & 0.67 & 0.18 & -0.13 & -1.16 & 0.13 & 1.96 & -0.67 \\ 0 & 0 & 0 & 0 & 0.67 & -0.98 & 0.13 & -1.16 & -0.13 & 0.18 & -0.67 & 1.96 \end{bmatrix} 10^{-9} \tag{4.4.8.14}$$

$$F = \begin{bmatrix} 0 \\ -6.25 \\ 0 \\ -6.25 \\ 0 \\ -12.5 \\ 0 \\ -12.5 \\ 0 \\ -6.25 \\ 0 \\ -6.25 \end{bmatrix} 10^{-3} N \tag{4.4.8.15}$$

17.1.4 (d)

With the global system assembled, we apply the essential boundary conditions to find the unknown nodal displacements and support reactions. This is achieved by partitioning the global system $\boldsymbol{KU} = \boldsymbol{F}$ into free (subscript 1) and constrained (subscript 2) degrees of freedom:

$$\begin{pmatrix} \boldsymbol{K}_{11} & \boldsymbol{K}_{12} \\ \boldsymbol{K}_{21} & \boldsymbol{K}_{22} \end{pmatrix} \begin{pmatrix} \boldsymbol{U}_1 \\ \boldsymbol{U}_2 \end{pmatrix} = \begin{pmatrix} \boldsymbol{F}_1 \\ \boldsymbol{F}_2 \end{pmatrix} \tag{4.4.8.16}$$

The unknown displacements $\boldsymbol{U}_1$ are found by solving the first set of equations:

$$\boldsymbol{U}_1 = \boldsymbol{K}_{11}^{-1}(\boldsymbol{F}_1 - \boldsymbol{K}_{12}\boldsymbol{U}_2) \tag{4.4.8.17}$$

Once the displacement field is fully known, the unknown reaction forces $\boldsymbol{F}_2$ at the supports are recovered using the second set of equations:

$$\boldsymbol{F}_2 = \boldsymbol{K}_{21}\boldsymbol{U}_1 + \boldsymbol{K}_{22}\boldsymbol{U}_2 \tag{4.4.8.18}$$

Although possible, the realocation of matrices can become very heavy on the computer memory. Another method, much simples programatically, is to enforce the known displacements into the matrix for all given supports, like done in the snipped below

```
# true if there is an imposition
boundary_vector = [
    true;
    true;
    false;
    true;
    false;
    false;
    false;
    false;
    false;
    false;
    false;
    false;
```

```
]

function apply_boundary(boundary_vector, K, F)
    K_sys, F_sys = copy(K), copy(F)
    for i in 1:length(F)
        if boundary_vector[i]
            K_sys[i, :] .= 0
            K_sys[:, i] .= 0
            K_sys[i, i] = 1
            F_sys[i] = 0
        end
    end
    return K_sys, F_sys
end

K_sys, F_sys = apply_boundary(boundary_vector, K, F)
U = K_sys \ F_sys

Reactions = K * U - F
;
```

Thus,

$$U = \begin{bmatrix} 0 \\ 0 \\ 0.5224 \\ 0 \\ 0.4174 \\ -1.8622 \\ 0.1049 \\ -1.8622 \\ 0.3125 \\ -2.5 \\ 0.2099 \\ -2.5 \end{bmatrix} 10^{-6} m \qquad \mathrm{R} = \begin{bmatrix} 0 \\ 5000 \\ 0 \\ 5000 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \mathrm{kN} \tag{4.4.8.19}$$

17.1.5 (e)

Once the global displacement vector $\boldsymbol{U}$ is found, the displacement vector for a specific element is recovered using its location vector $\ell^{(e)}$:

$$\boldsymbol{U}^{(e)} = \boldsymbol{U}\left(\ell^{(e)}\right) \tag{4.4.8.20}$$

To find the strains and stresses within the element, we evaluate them at the numerical integration points. The strain vector $\boldsymbol{\varepsilon}_i$ at an integration point i is obtained by multiplying the strain-displacement matrix $\boldsymbol{B}_i$ by the element displacement vector. Subsequently, the stress vector $\boldsymbol{\sigma}_i$ is computed using the material's constitutive matrix $\boldsymbol{D}$:

$$\boldsymbol{\varepsilon}_i = \boldsymbol{B}_i \boldsymbol{U}^{(e)} \tag{4.4.8.21}$$

$$\boldsymbol{\sigma}_i = \boldsymbol{D}\boldsymbol{\varepsilon}_i \tag{4.4.8.22}$$

The following script will extract the nodal displacements for element 1, evaluate the $\boldsymbol{B}$ matrix at its respective integration points, and compute the resulting strain and stress vectors.

```
function get_strain_and_stress(gim, D, U)
    quad_2 = [
        (-1 / sqrt(3), 1.0),
```

```
        ( 1 / sqrt(3), 1.0)
    ]

    # 3. Get displacements, strains, and stresses for nodes of interest
    for (elem_idx, d) in enumerate(gim)
        C = d["coords"]
        nodes = d["nodes"]

        dofs = Int[]

        for n in nodes
            push!(dofs, 2n - 1) # ux
            push!(dofs, 2n )    # uy
        end

        # Corrected: iterate directly over the values inside `dofs`
        Ulocal = [U[i] for i in dofs]
        d["strains"] = []
        d["stresses"] = []

        # 4. Loop over integration points to compute Strain and Stress
        for (xi, wi) in quad_2
            for (eta, wj) in quad_2

                # Evaluate B at this integration point
                B = get_B(C, xi, eta)

                # Compute strain and stress
                strain = B * Ulocal
                stress = D * strain

                push!(d["strains"], (xi, eta) => strain)
                push!(d["stresses"], (xi, eta) => stress)


            end
        end
    end
end

get_strain_and_stress(global_index_mapper, get_D(E, ν), U)


println("---- (ξ, η) => ϵ ----")
for i in global_index_mapper[1]["strains"]
    println(i)
end

println("---- (ξ, η) => σ ----")
for i in global_index_mapper[1]["stresses"]
    println(i)
end;
```

```
---- (ξ, η) => ϵ ----
(-0.5773502691896258,       -0.5773502691896258)       =>       [4.78033493734919e-7,
-1.8621735074626867e-6, 6.058946388417328e-8]
(-0.5773502691896258,       0.5773502691896258)       =>       [3.5685456596657403e-7,
-1.8621735074626867e-6, 6.058946388417185e-8]
(0.5773502691896258,       -0.5773502691896258)       =>       [4.78033493734919e-7,
-1.8621735074626882e-6, -6.058946388417169e-8]
```

```
(0.5773502691896258,       0.5773502691896258)       =>       [3.5685456596657403e-7,
-1.8621735074626882e-6, -6.05894638841729e-8]
---- (ξ, η) => σ ----
(-0.5773502691896258, -0.5773502691896258) => [266.45582654394275, -37176.85619261774,
484.7157110733862]
(-0.5773502691896258, 0.5773502691896258) => [-2318.6946325140825, -37823.14380738225,
484.7157110733748]
(0.5773502691896258, -0.5773502691896258) => [266.45582654393365, -37176.85619261778,
-484.7157110733735]
(0.5773502691896258, 0.5773502691896258) => [-2318.6946325140916, -37823.14380738229,
-484.71571107338326]
```

17.1.6 (f)

Finally, we may compare the analitical solution for the weight of the element, which is simply $\sigma = h\gamma$

```
# we can use the shape functions to get the
# coordinates for the lower integration point
# (ξ, η) = -1/sqrt(3) * (1, 1)

xi_lower, eta_lower = -1/sqrt(3), -1/sqrt(3)
x, y = C1' * N(xi_lower, eta_lower)
depth = 2.0 - y

analytical_stress = γ * depth

numerical_stress_vector = first(val for (coords, val) in global_index_mapper[1]
["stresses"] if coords == (xi_lower, eta_lower))

numerical_sigma_yy = numerical_stress_vector[2]

println("Analytical σ_yy: ", analytical_stress)
println("Numerical σ_yy:  ", abs(numerical_sigma_yy))
```

```
Analytical σ_yy: 44716.878364870325
Numerical σ_yy:  37176.85619261774
```

As we can see, the solutions differ considerably. This may be attributed mainly to the overly generous mesh, which creates errors naturally.

18. Exercise 4.6.3

Find all nodal displacements in the truss using the penalty method and the Lagrange multiplier method. Consider EA = 600 kN, P = 10 kN, and α = 30°. The lengths of elements 1, 2, and 3 are 3 m, 4 m, and 5 m, respectively. Compare the results.

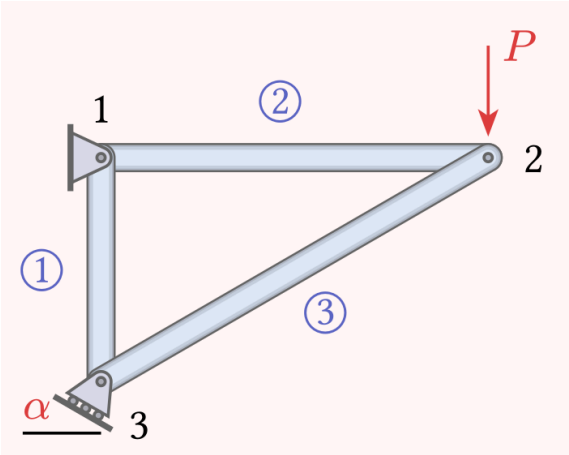

Figure 18.1: Truss with inclined support

18.1 Solution

There are 3 Constrains, thus b is a 3x1 vector. For 6 degrees of freedom (displacements), thus A must be a 3x6 matrix:

$$\boldsymbol{AU} = \boldsymbol{b} \tag{4.6.3.1}$$

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -\sin(\alpha) & \cos(\alpha) \end{pmatrix} \begin{pmatrix} u_{1x} \\ u_{1y} \\ u_{2x} \\ u_{2y} \\ u_{3x} \\ u_{3y} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \tag{4.6.3.2}$$

The following code then solves the problem above defined, using the Direct Stiffness Method.

```
using LinearAlgebra

    function solve_truss()
        # Problem Parameters
        EA = 600.0          # kN
        P = 10.0            # kN
        # alpha is negative because the support rotated clockwise
        alpha = -30       # degrees

        # Nodal Coords
        nodes = [
            0.0   0.0;
            0.0   4.0;
```

```
    -3.0   0.0;
    ]

    # Element Connectivity (Node A, Node B)
    elements = [
        (1, 3), # Element 1
        (1, 2), # Element 2
        (3, 2)  # Element 3
    ]

    ndofs = 2 * size(nodes, 1)
    K = zeros(ndofs, ndofs)
    F = zeros(ndofs)

    # Apply external load P downwards at Node 2
    F[4] = -P

    for (i, j) in elements
        dx = nodes[j, 1] - nodes[i, 1]
        dy = nodes[j, 2] - nodes[i, 2]
        L = sqrt(dx^2 + dy^2)

        c = dx / L
        s = dy / L

        # Local element stiffness matrix
        k_local = (EA / L) * [
            c^2   c*s  -c^2  -c*s
            c*s   s^2  -c*s  -s^2
            -c^2  -c*s   c^2   c*s
            -c*s  -s^2   c*s   s^2
        ]

        # Map to global DOFs
        dofs = [2*i-1, 2*i, 2*j-1, 2*j]
        for r in 1:4
            for c_idx in 1:4
                K[dofs[r], dofs[c_idx]] += k_local[r, c_idx]
            end
        end
    end

    # Setup Constraint Equations (A * U = b)
    # We have 3 constraints: u1x = 0, u1y = 0, and the roller at Node 3
    A = zeros(3, ndofs)
    b = zeros(3)

    A[1,1] = 1.0 # (u1x = 0)

    A[2, 2] = 1.0  #(u1y = 0)

    # Constraint 3: Node 3 Inclined Roller (DOFs 5 and 6)
    nx = -sind(alpha)
    ny = cosd(alpha)
    A[3, 5] = nx  # u3x component
    A[3, 6] = ny  # u3y component

    # ------------------------------------------------------------
    # Penalty Method
    # ------------------------------------------------------------
```

```
        display("=== Penalty Method ===")
        pen_val = 1e8 * maximum(K)

        K_pen = K + pen_val * (A' * A)
        F_pen = F + pen_val * (A' * b)

        U_pen = K_pen \ F_pen

        display("Displacements (U):")
        display(round.(U_pen, digits=6))

        # ------------------------------------------------------------
        # Lagrange
        # ------------------------------------------------------------
        display("=== Lagrange Multiplier Method ===")
        K_lag = [K A'; A zeros(3, 3)]
        F_lag = [F; b]

        X = K_lag \ F_lag

        U_lag = X[1:ndofs]
        Lambda = X[ndofs+1:end]

        display("Displacements (U):")
        display(round.(U_lag, digits=6))

        display("Lagrange Multipliers (Constraint Forces):")
        display(round.(Lambda, digits=6))

    end

    solve_truss()
```

```
"=== Penalty Method ==="
```

```
"Displacements (U):"
```

```
6-element Vector{Float64}:
  0.0
 -0.0
  0.088889
 -0.066667
  0.0
 -0.0
```

```
"=== Lagrange Multiplier Method ==="
```

```
"Displacements (U):"
```

```
6-element Vector{Float64}:
  0.0
 -0.0
  0.088889
 -0.066667
  0.0
 -0.0
```

```
"Lagrange Multipliers (Constraint Forces):"
```

```
3-element Vector{Float64}:
   0.0
 -10.0
  -0.0
```

19. Exercise 4.6.4

In the symmetric truss below, element 3 is rigid. For the other elements, EA = 600 kN. Find all nodal displacements using the Lagrange multiplier method. Consider the length of element 1 equal to 2 m, α = 45°, and P = 10 kN.

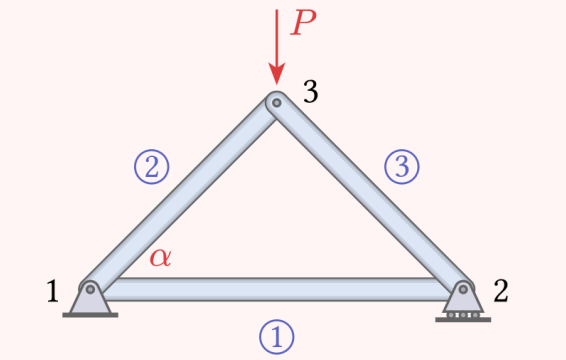

Figure 19.1: Truss with inclined rigit element

19.1 Solution

```
using LinearAlgebra

function solve_truss()
    # Problem Parameters
    EA = 600.0        # kN
    P = 10.0          # kN

    # Nodal Coords (x, y)
    nodes = [
        0.0   0.0;  # Node 1
        2.0   0.0;  # Node 2
        1.0   1.0;  # Node 3
    ]

    # Element Connectivity (Node A, Node B)
    # third element is non-elastic
    elements = [
        (1, 2), # Element 1
        (1, 3)  # Element 2
    ]

    ndofs = 2 * size(nodes, 1)
    K = zeros(ndofs, ndofs)
    F = zeros(ndofs)

    F[6] = -P

    for (i, j) in elements
        dx = nodes[j, 1] - nodes[i, 1]
        dy = nodes[j, 2] - nodes[i, 2]
```

```
        L = sqrt(dx^2 + dy^2)

        c = dx / L
        s = dy / L

        k_local = (EA / L) * [
             c^2   c*s  -c^2  -c*s
             c*s   s^2  -c*s  -s^2
            -c^2  -c*s   c^2   c*s
            -c*s  -s^2   c*s   s^2
        ]

        dofs = [2*i-1, 2*i, 2*j-1, 2*j]
        for r in 1:4
            for c_idx in 1:4
                K[dofs[r], dofs[c_idx]] += k_local[r, c_idx]
            end
        end
    end

    # (A * U = b)
    A = zeros(4, ndofs)
    b = zeros(4)

    A[1, 1] = 1.0

    A[2, 2] = 1.0

    A[3, 4] = 1.0

    dx3 = nodes[3, 1] - nodes[2, 1]
    dy3 = nodes[3, 2] - nodes[2, 2]
    L3 = sqrt(dx3^2 + dy3^2)

    c3 = dx3 / L3
    s3 = dy3 / L3

    A[4, 3] = -c3
    A[4, 4] = -s3
    A[4, 5] =  c3
    A[4, 6] =  s3

    # ------------------------------------------------------------
    # Lagrange Multiplier Method
    # ------------------------------------------------------------
    display("=== Lagrange Multiplier Method ===")

    # Build the augmented block matrix
    K_lag = [K A'; A zeros(4, 4)]
    F_lag = [F; b]

    # Solve the augmented system
    X = K_lag \ F_lag

    # Extract displacements and multipliers (reactions / internal forces)
    U_lag = X[1:ndofs]
    Lambda = X[ndofs+1:end]

    display("Displacements (U):")
    display(round.(U_lag, digits=6))
```

```
    display("Lagrange Multipliers (Constraint Forces):")
    display(round.(Lambda, digits=6))
end

solve_truss()
```

```
"=== Lagrange Multiplier Method ==="
```

```
"Displacements (U):"
```

```
6-element Vector{Float64}:
 -0.0
  0.0
  0.016667
  0.0
 -0.003452
 -0.020118
```

```
"Lagrange Multipliers (Constraint Forces):"
```

```
4-element Vector{Float64}:
 -0.0
 -5.0
 -5.0
 -7.071068
```